パワー ユーザー ガイド

ProDG for Nintendo GameCube™

ビルド ツール

SN Systems Ltd
バージョン **3.9.3**
2003 年 **5** 月

著作権© 2001-2003 SN Systems Ltd. 無断転載を禁ず

"ProDG" および SN ロゴは、SN Systems Ltd の商標です。"GameCube" は、Nintendo Co. Ltd の商標です。"Microsoft"、"Visual Studio"、 "Win32"、"Windows"、"Windows NT"は、Microsoft Corporation の登録商標です。 "GNU" は Free Software Foundation の商標です。"Linux"は Linus Torvalds の 商標です。"Pentium" は Intel の登録商標です。この 文書で使用される商品名 または会社名は、それぞれの所有者の商標です。

NINTENDO GAMECUBE™

NINTENDO GAMECUBE は、任天堂株式会社の登録商標です。
Windows、WindowsNT は米国 Microsoft Corporation の米国およびその他の国に おける登録商標または商標です。製品および内容物の外見、形状、仕様は、予告なしに変更される場合があります。

ProDGNGC_PU(BT)-J / パワーユーザーガイド ProDG for Nintendo GameCube ビルドツール / 2003 年 5 月

ドキュメント変更履歴

Ver.	日付	変更
3.9.3	2003 年 5 月	ProDG for Nintendo GameCube v3.9.3 をリリース。v3.9.2 からの変更点: ソースブラウジングと VSI チャプターのディペンダンシーの 表示に関する追加情報; 多数の「トラブルシ ューティング」のセクションの追加;「ビルドの最適化」に関する新しい章の追加;「リロケータ ブルモジュール」に関する新しい 章に分割した情報; 'ユーティリティ' 章に新しく追加された blankelf と make; 非サポート アセンブラ/リンカ オプションおよびエラー メッセージが付属書に転載;コンプライヤ— C/C++ スイッチのテーブルと警告の修正;アセンブラ エラーと 警告メッセージ のフォーマットの訂正;-x<type> ngccc スイッチの修正;アセンブラ スイッチ テーブルの修正; #pragma mathwarn の追加;「追加されたコンパイラの 特徴」セクションの追加; 新しい ngcld スイッチ: -sn-full-map, -sn-overlays 及び -whole-archive;VSI の 章にセットアップのステップを追加; 新しい snarl MRI スクリ プティングドキュメンテーション; 最適化の章を改訂. '依存情報の生成'と'ソースのブラウジング'のセクションを改訂; "asm volatile"ステートメントの修正; SN_HOOK_FPE() マクロの追加

目次

第 1 章: はじめに 1

概要 1
ProDG Plus 1
テクニカル サポートとアップグレード 2
Web を経由した ProDG のアップデート 2
ドルフィン SDK をアップデートする 3

第 2 章: コマンドライン ビルド ツールの使用法 5

ProDG ビルド ツール 5
ProDG ユーティリティ 5
GNU ビルド ツール 6
sn.ini - SN System の設定ファイル 6
sn.ini で変数を使用する 7
libsn.a - SN Systems ライブラリ 7
浮上小数点例外のインターセプト 8
libsn.a 用バージョンの入手 8

第 3 章: Visual Studio 6.0 統合機能 9

Visual Studio 6.0 統合機能の概要 9
Visual Studio を設定する 9
Visual Studio で GameCube プロジェクトを作成する 9
アクティブプロジェクトの設定 11
Visual Studio 6.0 統合機能を設定する 12
GameCube プロジェクトをビルドする 13
プロジェクトファイルを追加する 13
プロジェクトをビルドする 13
ELF ファイルを起動/実行する 14
ProDG デバッガとの統合 15
プロジェクトにブレークポイントを設定する 15
Visual Studio でソースを編集する 16
テンポラリファイルを削除する 16
依存情報の生成 16
Visual C++ はどのようにして依存情報を生成するか 16
ソースのブラウジング 17
Visual C++ がブラウズ情報を生成する仕組み 17
NGC_in_VC.h ファイルを用いてブラウズ情報の生成機能を向上させる 17
ソースコードを修正してブラウズ情報の生成を向上させる 18
makefile をソースブラウジングに使う 18

トラブルシューティング 18
サードパーティ統合ブレークのインストール 18
「未定義シンボルへ参照」エラー 20

第 4 章: C と C++でのメモリ管理 21

序文 21
malloc/free に C++例外操作が抜けている場合 21
strdup (malloc のみ) 21
setvbuf (malloc/free) 21
malloc を直接呼出し 21
free を直接呼出し 22
malloc と free を使ったプログラミング 22
new と delete を利用した C++のプログラム 23
I/O ストリームバッファを使用する 24

第 5 章: ProDG コンパイラ ドライバ 27

ngccc コンパイラ ドライバの使用 27
ngccc コマンドライン書式 28
ngccc オプション 28
ファイル名 33
デフォルトファイル拡張子アクションを最優先する 34
ゲームキューブライブラリを指定する 34
応答ファイルを使用する 35
C プログラミングに関する考慮事項 35
グローバル変数のメモリ場所の指定 35
新 sync イントリンシック 35
Naked 関数のサポート 35
高速キャスティング 36
実行 36
プログラミングにおける考慮事項 36
単一ペア浮動小数点演算 37
タイプの実装 37
変換 38
ネイティブ PS 演算 39
SN API for PS (SNAPS) 演算 39
一般的なプログラミングにおける考慮事項 41
例 42
ゲームキューブプログラミングのヒント 45
トラブルシューティング 46
警告を発しない C コンパイラ 46
「未知のエスケープシーケンス」エラー 46

第 6 章: ProDG アセンブラ 49

SN Systems のアセンブラを使用する 49
コマンドライン書式 49
アセンブラ命令 50

SN Systems 命令 51
レジスタ名称指定およびレジスタフィールド 51
範囲の指定 52
アドレスモード 53
入力ファイル 53
ファイル名と行番号 53
出力(オブジェクト)ファイル 54
エラーおよび警告メッセージ 54
シンボル名 54
ローカルシンボル 55
レジスタリファレンス 55
レジスタ数 56
レジスタ名 56
レジスタ名とインラインアセンブラパラメータ 56
レジスタとインプット/出力/クラバーリスト 56
インラインアセンブラのデバッギング 57

第 7 章: ProDG リンカ 59

ngcld リンカを使用する 59
必要メモリ 59
リンカ スクリプト 59
セクション 60
リンカスクリプトでファイルを参照する 60
ngcld コマンドライン書式 60
ngcld スイッチ 61
小データ領域のサポート 62
小データを使用する目的 63
小データセクションの適用範囲を調節する 63
トラブルシューティング 64
「未定義シンボルへの参照」エラー 64

第 8 章: ビルドの最適化 65

はじめに 65
ELF のサイズ最適化 65
コンパイラ オプション 65
リンカ オプション 66
シンボル情報の削除 67
不必要なデバッグ情報の削除 67
重複する非インライン関数の生成防止 67
グローバル デストラクタの削除 67
実行速度向上のための最適化 68
コンパイラの最適化スイッチの使用方法 68
ビルド速度の最適化 68
不必要なインクルード ファイルの排除 68
システム リソースの最大化 69
テクニカルサポート 70
SN Systems へのフィードバック 70

第 9 章: 再配置可能モジュール **71**

再配置可能モジュールをビルドする 71

第 10 章: ユーティリティ **75**

blankelf - blank ELF ユーティリティ 75
make 75
snarl - SN アーカイヴライブラリアン 76
ライブラリの読み込み 76
クロスプラットフォーム 76
コマンドライン書式 76
Verbose モード 78
アーカイヴファイルの印刷 78
オブジェクトファイルの印刷 78
シンボル表の表示 78
高速追加 78
複数回定義されたシンボルの警告 79
シンボル操作コマンド 80
応答ファイルスクリプティング 80
MRI スクリプティング 80
MRI スクリプトをファイルから実行する 82

第 11 章: Gekko プロファイリング用ビルド **85**

ゲームの中の Gekko プロファイリングをサポートするには 85
Gekko プロファイラ API 85

別表 A: アセンブラ参考資料 **89**

サポートされていないアセンブラオプション 89
サポートされていないアセンブラダイレクティブ 89

別表 B: リンカ参考資料 **91**

サポートされていない ld スイッチ 91
ngcld でサポートしていないスクリプトファイル命令 92
エラーメッセージ 93
共通エラーメッセージ 93
ngcld に特有のエラーメッセージ 95

インデックス **97**

第 1 章: はじめに

概要

ProDG は、ニンテンドーゲームキューブ をコンソールとするゲームのビルドやデバッグを行なうための、SN Systems 製フル装備 Win32 ツール群です。ProDG ツールを使 用すれば、 Win32 PC 上ですぐにアプリケーションをビルドし、ニンテンドーゲームキュー ブの開発ハードウェアにアップロードして、実行/デバッグできます。

ProDG の構成は下記のとおりです:

- **ProDG ビルドツール**: Microsoft Visual Studio インテグレーションオプション; ProDG コンパイラドライバ; ProDG リンカ; SN Systems ユーティリティとライブラリ; ProDG アセンブラ。
- **ProDG Target Manager** を使うと、ネットワーク内で GameCube ターゲットへの接続をコントロールできます。詳細は、*ProDG Target Manager の ユーザーガイド*を参照してください。
- **ProDG Debugger** は、GameCube アプリケーションのデバッグ に必要な完璧な機能を備えた Win32 デバッガです。デバッガに関する詳細は、*Power User's Guide to ProDG for Nintendo GameCube Debugger* をご覧く ださい。

このマニュアルは、ProDG ビルドツールに関しての説明です。

ProDG Plus

ProDG Plus は、GameCube ゲーム開発/デバッグ用の高性能ツール パッケージです。

- **ProDG** — このパッケージには、C/C++ コンパイラ、アセンブラ、 リンカ、デバッガ、ターゲット マネージャが含まれます。またオプションの Visual Studio 統合機能 には、実行ファイルやライブラリのビルド、および Visual Studio 内から ProDG Debugger を起動するための App-Wizards が含まれます。
- **ProDG Debugger の高度な機能** — デバッガのスクリプト機能や、強化されたその他の機能により、ProDG Plus ユーザーにとって GameCube デバッガの新しい標準となります。
- **Tuner** — プログラムの動作をキャプチャして視覚化することにより、コードに おけるコンフリクトや問題の解決を支援します。このツールにより、ハイパフォーマンスなゲームを「手探り」なしで、プログラムすることが可能になります。またホスト PC へのデータ キャプチャは、ゲームのプレイと同時にリアルタイム で行われま

す。キャプチャしたデータは、フレーム単位で分析するこ とが可能で、保存したデータと最適化したコードと比較することもできます。

テクニカル サポートとアップグレード

当社 Web サイトのサポート エリアには、SN Systems の全製品に関する各種の 最新サポートが用意されています。このページには、SN Systems のユーザー ID とパスワードを所有する登録ユーザーのみがアクセスできます。

- ユーザー ID やパスワードを忘れた場合は、電子メールで **webmaster@snsys.com** までご連絡ください。

www.snsys.com/support (英語)
www.snsys.jp/support (日本語)

サポート エリアで解決法を見つけられない場合は、当社サポート チーム (下記アドレス) まで電子メールでご連絡ください。

support@snsys.com (英語)
j-support@snsys.com (日本語)

ご連絡の際は、発生している問題を明記し、ご使用のソフトウェア バージョンお よびハードウェア環境の詳細も記載してください。SN Systems サポートのログ番号 (LN 番号) をお持ちの場合は、問題に関するすべての電子メールにその 番号を記載してくだ さい。

SN Systems Limited
4th Floor - Redcliff Quay
120 Redcliff Street
Bristol BS1 6HU
United Kingdom

電話:+44 (0)117 929 9733
電話:+44 (0)117 929 9251

URL:**www.snsys.com** (英語)
www.snsys.jp (日本語)

Web を経由した ProDG のアップデート

ProDG のアップデートは、SN Systems の Web サイトに定期的にアップロードされます (2 ページの「テクニカル サポートとアップグレード」を参照)。ダウンロードのページにア クセスするには、[テクニカル サポート]、[ダウンロード] を選択します。このページにアク セスするには、SN Systems のユーザー ID とパスワードが必要です。

- ProDG の製品に関する全面的なアップデートは、**[フルインストール]** のセクションに表示されます。

- 十分テストされたコンポーネントに関するアップデートは、**[プロダクトアップグレード]** のセクションに表示されます。

- Beta バージョンのコンポーネントに関しては、**[プリ・リリース]** のセクションに表示されることがあります。

ドルフィン SDK をアップデートする

認証を受けたゲームキューブ開発者は、任天堂株式会社から直接ドルフィン SDK をアップデートします。ドルフィン SDK のマイナーアップグレードに際しては、ProDG の再インストールは通常必要ありません。

- 任天堂がドルフィン SDK の新バージョンを大幅に変更した場合は、SN Systems から ProDG の互換ビルドが提供されるまでインストールしないで下さい。現時点でサポートされているドルフィン SDK のバージョンは、SN Systems のウェブサ イトに記載されています。

[このページは意図的に空白にされています]

第 2 章: コマンドライン ビルド ツールの使用法

ProDG ビルド ツール

インストーラは、以下のようなディレクトリツリーを作成します。

.\ngc	Readme、ProDG 設定、バージョンユーティリティ
.\ngc\bin	実行可能ファイル
.\ngc\include	標準 C ライブラリヘッダファイル
.\ngc\include\machine	標準 C ライブラリヘッダファイル s
.\ngc\include\sys	標準 C ライブラリヘッダファイル
.\ngc\lib	リンクオブジェクトライブラリ
.\ngc\sndemo	SN Systems デモ
.\ngc\stl	標準テンプレートライブラリ

GameCube で使用が可能な ProDG ビルドツールの詳細を、それぞれの 機能別に、以下の表で解説します。

命令	解説
ngccc	ProDG Nintendo GameCube のコンパイラ ドライバ、27 ページの「ngccc コンパイラ ドライバの使用」を参照。
ngcas	ProDG Nintendo GameCube のアセンブラ、49 ページの「SN Systems のアセンブラを使用する」を参照。
ngcld	ProDG Nintendo GameCube のリンカ、59 ページの 「ngcld リンカを使用する」を参照。

ProDG ユーティリティ

以下の ProDG ユーティリティもご利用になれます:

blankelf	Blank ELF ユーティリティ、75 ページの"blankelf - blank ELF ユーティリティ" を参照。
make	エグゼクタブルの発生とプログラムの他のファイルをコントロールするツール;make は、デフォルト名の makefile を使って、コマンド ファイルからのインプットを取得、75 ページの「make」を参照。

snarl	SN アーカイヴライブラリアン、76 ページの snarl - SN アーカイヴライブラリアン」を参照。SN Archive Librarian.

GNU ビルド ツール

ProDG は、GNU C および C++コンパイラと各種 GNU ユーティリティの GameCube 版が含まれた形で提供されます。

Nintendo GameCube で使用が可能な GNU ビルドツールの詳細を、それぞれの 機能別に、以下の表で解説します。各々の GNU ツールの使用法に関す る詳細は、GNU の Web サイト **www.gnu.org**. でオンライン ドキュメントを参照してください。

命令	解説
cc1	強化版 GNU C コンパイラ。通常、ngccc などの コンパイラドライバにより呼び出される。ccl に送られるファイルは、まず cpp で前処理する必要あり。
cc1plus	強化版 GNU C++コンパイラ。通常、gcc や ngccc などの コンパイラ ドライバにより呼び出される。cclplus に送られるファイルは、まず cpp で前処理する必要あり。
cpp	GNU コンパイラ前処理プログラム。通常、gcc や ngccc などのコンパイラドライバにより呼び出される。ccl、cclplus に送られるファイルは、まず cpp で前処理する必要あり。

sn.ini - SN System の設定ファイル

ProDG for Nintendo GameCube は、sn.ini という設定ファイルをインストールします。このファイルは、SN_NGC_PATH 環境変数で指定されたディレクトリに配置され、どの ProDG for Nintendo GameCube のコンポーネントが使われていいる かを特定するために使用されます。特に Visual Studio 統合機能 と ngccc コンパイラ ドライバで 有用な機能が発揮されます。

以下は sn.ini のデフォルトコンテンツです。

```
[ngccc]
compiler_path=<install dir>\bin
c_include_path=<install dir>\include
library_path=<install dir>\lib
assembler_path=<install dir>\bin
linker_path=<install dir>\bin
cplus_include path=<install dir>\include
linker_name=ngcld.exe
opt_assembler_name=ngcas
assembler_name=ngcas
linker_script=<install dir>\bin\ngc.ld
stdlib=<standard library list>
rel_link_file=<install dir>\bin\plfkeep.lst
rel_linker_script=<install dir>\bin\plf.ld
```

'#' で始まる行は、コメントとして扱われ無視されますから、'#' に続くどの sn.ini 行を変更するかによって、ビルドツール、アセンブラ、およびリンカの異なるバージョン間での スワップが簡単に可能となります。

sn.ini で変数を使用する

sn.ini ファイルでは、変数が使用でき、makefile の内部変数と同様に機能します。たとえば:

```
[ngccc]
DOLPHIN_INC=c:\DolphinSDK1.0\include
DOLPHIN_LIB=c:\DolphinSDK1.0\HW2\lib
PRODG=c:\ngc

#compiler_path=$(PRODG)\bin\gcc3
compiler_path=$(PRODG)\bin

c_include_path=$(PRODG)\include;$(DOLPHIN_INC);$(PRODG);$(DOLPHIN_INC)
\dolphin
library_path=$(PRODG)\lib
assembler_path=$(PRODG)\bin
linker_path=$(PRODG)\bin
cplus_include_path=$(PRODG)\include\stlport-4.5.3\stlport;
$(PRODG)\include;$(DOLPHIN_INC);$(PRODG)
#cplus_include_path=$(PRODG)\bin\gcc3;$(PRODG)\include;$(DOLPHIN_INC)

linker_name=ngcld
rel_link_file="$(PRODG)\lib\plf keep.lst"
opt_assembler_name=ngcas
assembler_name=ngcas
linker_script=$(PRODG)\bin\ngc.ld
rel_linker_script=$(PRODG)\bin\plf.ld
stdlib=libsn.a libc.a libm.a libgcc.a libstl.a $(DOLPHIN_LIB)/si.a
$(DOLPHIN_LIB)/exi.a $(DOLPHIN_LIB)/base.a $(DOLPHIN_LIB)/os.a
$(DOLPHIN_LIB)/db.a $(DOLPHIN_LIB)/mtx.a $(DOLPHIN_LIB)/dvd.a
$(DOLPHIN_LIB)/vi.a $(DOLPHIN_LIB)/demo.a $(DOLPHIN_LIB)/pad.a
$(DOLPHIN_LIB)/ai.a $(DOLPHIN_LIB)/ar.a $(DOLPHIN_LIB)/ax.a
$(DOLPHIN_LIB)/dsp.a $(DOLPHIN_LIB)/card.a $(DOLPHIN_LIB)/hio.a
$(DOLPHIN_LIB)/mcc.a c:/gameOptixDDK/lib/HW2/amcExi2.a
c:/NPDP/GDEV/lib/OdemuExi2.a $(DOLPHIN_LIB)/gx.a $(DOLPHIN_LIB)/G2D.a
$(DOLPHIN_LIB)/texPalette.a $(DOLPHIN_LIB)/fileCache.a
$(DOLPHIN_LIB)/support.a $(DOLPHIN_LIB)/perf.a
```

この設定では、sn.ini の開始時に COMPILER_(X) 変数を変更するだけで、コンパイラを変更できます。

当然、環境変数も使用できます。たとえば:

```
Assembler_path=%SN_NGC_PATH%\bin
```

libsn.a - SN Systems ライブラリ

libsn.a アーカイヴは、SN Systems で作成された Gekko ターゲット・システム コードの主要な保管場所です。これには、DLL サポートとプロファイリングコードが含 まれます。Gekko libs リストの最初に、必ず libsn.a が含まれるようにし てください。これは、SN Systems では、ここにその他のターゲット・システムコードが必要に応じて保 管される からです。

浮上小数点例外のインターセプト

浮上小数点例外は、通常 OS に連鎖されますが、SN_HOOK_FPE()マクロをインクルードする事により ProDG デバッガが浮上小数点例外をインターセプトするよう強制する事ができます。

libsn.a 用バージョンの入手

__SN_Libsn_バージョン グローバル シンボルを使うと、実行時にライブラリバージョンの入手が可能になります。 例:

```
printf("libsn version = %d\n",__SN_Libsn_version[0]);
```

第 3 章: Visual Studio 6.0 統合機能

Visual Studio 6.0 統合機能の概要

本章は ProDG を Microsoft Visual Studio と統合する方法について解説します。ProDG を正しくインストールし、Visual Studio 統合を選択すると、Visual Studio の 中で GameCube プロジェクトを生成することができます。そして、GameCube プロジェクトをビルドし、ProDG デバッガを用いてデバッグします。

Visual Studio 6.0 統合機能の主な特徴は、以下の通りです。

- ProDG コンパイラドライバとリンカを用いてプロジェクトをビルドする。
- Visual Studio ソースを使用してプロジェクトを閲覧する
- コンパイラやリンカのエラーや警告を Visual Studio 形式で出力し、出力ウィンドウのビルドエラーをダブルクリックすれば、該当する行のソースファイルを Visual Studio エディタで開けるようになる
- ProDG デバッガを Visual Studio から直接呼び出して、プロジェクトをデバッグ可能にする
- ProDG デバッガのデバッグセッション開始／終了時点で、Visual Studio の ブレークポイントをインポート、エクスポートする
- デバッガのソースペインに表示されているソースを Visual Studio エディタで 開き、ソースファイルを編集可能にする

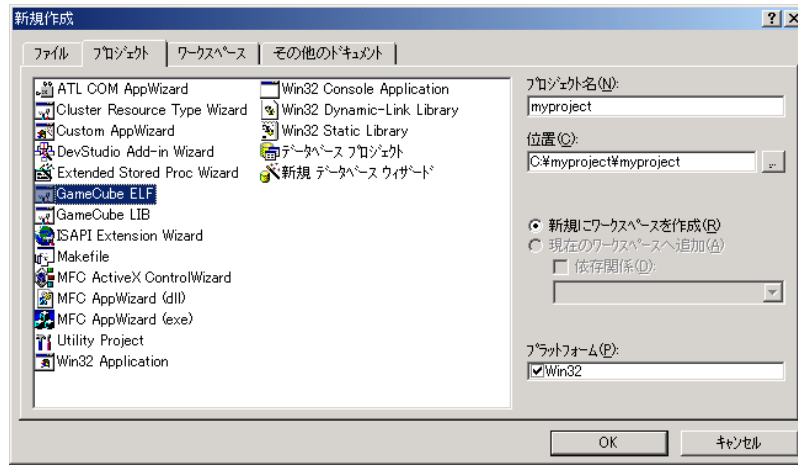
Visual Studio を設定する

Visual Studio をインストールした後、最初に起動した際には、**Tools** メニューの **Customize** をクリックし **Add-ins and Macro Files** タブを選びます。そこで現れるダイアログで **ProDG for GameCube Developer Studio Add-in** を選んでこれをイネーブルする必要があります。

Visual Studio で GameCube プロジェクトを作成する

設定ステップが完了すれば、以下の手順で Visual Studio を使用して新規 GameCube プロジェクトの作成が可能になります。

1. **[ファイル]** メニューの **[新規]**をクリックして、**[プロジェクト]**タブをクリック すると、以下のダイアログが表示されます。

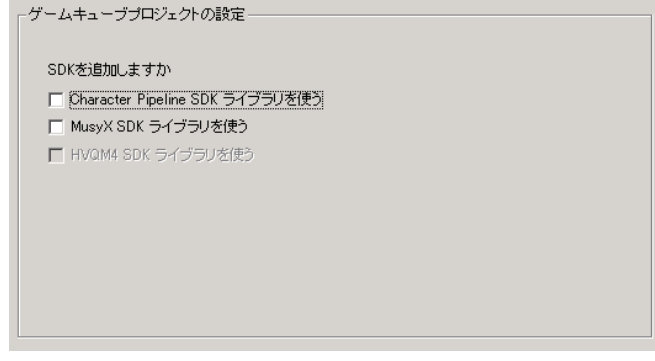


2. GameCube AppWizard である **GameCube ELF** を選びます(ここでは **GameCube ELF** AppWizard を一例としていますが、**GameCube LIB** と **GameCube DLL** AppWizards も同様に行ないます。ただしそ ちらではステップ 5 にある GameCube Project Setup のダイアログは出 てきません)

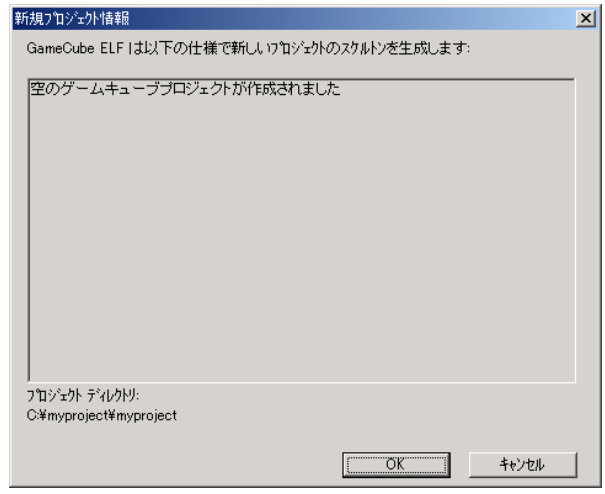
3. **プロジェクト名**フィールドに新しいプロジェクトの名称を入力します。

4. **位置**フィールドで、新しいプロジェクトを保存したい場所を参照し、**OK** をクリックします。

5. ここで現れるダイアログで **Character Pipeline SDK**、**MusyX SDK** あるいは **HVQM4 SDK** から提供される追加のライブラリ を使うかどうかを選択 します。ローカルマシンにインストールされていない SDK は選択することはできません。



6. 新しいプロジェクトが正しく作成され、Visual Studio でプロジェクトが 開かれ たことを確認する以下のようなダイアログが表示されたら、**OK** をクリックします。



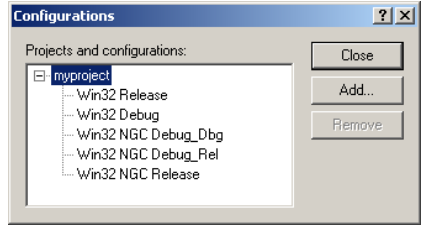
7. パスを含む標準 SDK はデフォルトで新しいプロジェクトに追加されていますが、プロジェクトのソースファイルが Visual Studio のプロジェクトファイルと別の 場所に ある場合、ソースファイルと同じディレクトリにあっても、Visual Studio は SDK を含まないファイルを見つけられません。この問題を解決するには、以下の 2 つの方法のどちらかを実行します。

   - (全てのプロジェクトについて該当する場合)**ツール**メニューの[オプション] ダイアログにある**ディレクトリ**タブで、プロジェクトのヘッダファイルを含む ディレクトリを、収録ファイルのパスに追加する。

   - (現在のプロジェクトについてのみ該当する場合)**プロジェクト**メニューの **設定**をクリックして **C/C++**タブを選択し、**カテゴリ**:**プリプロセッサ** を選択した状態で**インクルードファイルのパス**テキストボックスに追加するディレクトリを設定する。

アクティブプロジェクトの設定

各々の GameCube プロジェクトでデバッグおよびノンデバッグビルド設定が生成されます。プロジェクトとそのコンフィグレーションを表示するには、**Build** メニューオプションから **Configurations** を選びます。**GameCube ELF** には 3 種類のビルド構成があります。**Win32 NGC Debug_Dbg** (Nintendo ライブラリのリリース版を 用いてビル ドされる デバッグビルド)、**Win32 NGC Debug_Rel Dbg** (Nintendo ライブラリの リリース 版を用いてビルドされるデバッグビルド)、**Win32 NGC Release** (Nintendo ラ イブラリのリリース版を用いてビルドされるリリースビルド)。**Win32 Release** と **Win32 Debug** オプションは自動的に生成されますので意識する必要はありません。

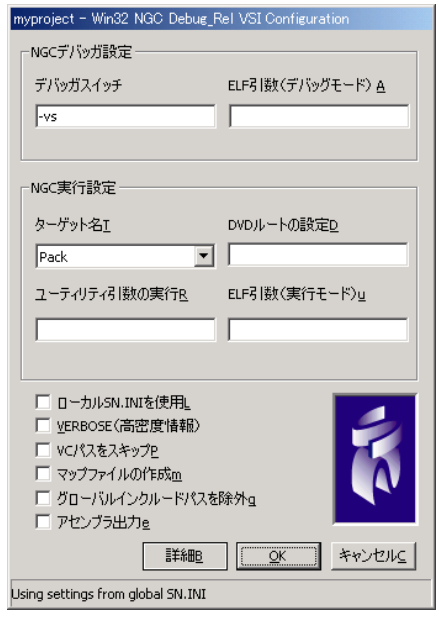


アクティブプロジェクトの設定は、**ビルド**メニューから **[この設定をアクティブにする]** を選択して設定します。

Visual Studio 6.0 統合機能を設定する

Visual Studio 統合機能の動作は、プロジェクトごとに設定できます。これの 設定には、新規プロジェクトを作成した後、下図のような [**VSI コントロールパネル**] ツールバーボタンをクリックします。

すると、以下のような [VSI 設定] ダイアログが表示されます。



NGC デバッガ 設定

コマンドラインスイッチや、ProDG デバッガのプログラムである ngcdbg に引き渡される ELF コマンドラインの引数を設定できます。

> **注:** 不正なスイッチを設定すると、ProDG デバッガが予 想外の動作をする場合があります。デフォルトの値を変更するときは、特に注意して下さい。

NGC 実行設定

ProDG コマンドラインユーティリティ、ngcrun にコマンドライン スイッチおよび ELF コマンドライン引数を引き渡すよ う設定できます。

ローカル SN.INI を使用

Visual Studio 統合機能はデフォルトで SN_NGC_PATH (環境変数) が指定するディレクトリにある sn.ini から環境を 取得します(6ページ「sn.ini - SN System の設定ファイル」参照)。しかし、この チェックボックスをオンにす れば、ローカルの sn.ini ファイルを使用して環境を取得できます。

	また、ローカルの sn.ini ファイルがない場合は、新たに作成可能です。
Verbose	コンパイル中に詳細を出力できます。
VC パスをスキップ	デフォルトでは、C および C++のファイルは、Visual Studio 統合機能で 2 度コンパイルされます。最初は Visual C コンパイラによって依存度チェックのために行なわれ、2 度目は SN Systems のコンパイラドライバである ngccc によって行なわれます (27 ページ「ngccc コンパイラ ドライバの使用」参照)。このオプションをオンにすると、Visual C コンパイラによるコンパイルを省略 できるため、コンパイル作業を高速化できます。
マップファイルの作成	リンカで mapfile を生成するには、このオプションをチェックします
グローバルインクルードパスを除外	このオプションをチェックすると、グローバル収録パス(**ツール → オプション → ディレクトリ → インクルード** 画面で一覧表示) の収録ファイルを検索せずに、プロジェクトのディレクトリのみを検索するように設定できます。これにより、グローバルヘッダファイルの機種依存コードからプロジェクトを完全に区別できます。
アセンブラ出力	コンパイラにアセンブラリストを保存するには、このオプションをチェックします

sn.ini ファイルのプロジェクト設定を保存する場合は **OK** を、中止する場合は**キャンセル**をクリックします。

GameCube プロジェクトをビルドする

Visual Studio 統合機能で、通常 Visual Studio を使用するのと同様 GameCube プロジェクトをビルドできます。本セクションではその方法を説明します。

プロジェクトファイルを追加する

GameCube ELF の新しいプロジェクトはリンカスクリプト ngc.ld で初期化されます。

プロジェクトソースファイルを、**プロジェクト**メニューの**プロジェクト追加 >ファイル** コマンドを使用して新規プロジェクトに追加する必要があります。プロジェクトソースファイルはプロジェクト makefile (.c、.s など)に不可欠であり、プロジェクトをビルドする 前に全てを追加する必要があります。

プロジェクトをビルドする

プロジェクトのビルドは、通常の操作と同様、Visual Studio の**ビルド**メニューから **クリーン**、**コンパイル**、**ビルド**などのコマンドを呼び出して実行します。

下の表は、Visual C のコンパイラスイッチと、それが GNU ではどのように表現されるかを一覧表示したものです。

VC	GNU	意味
/D	-D	プリプロセッサの定数を定義する
/debug	-g	デバッグ情報
/Fi	-include	ファイル名を追加する
/Fo	-o	ファイル名を出力する
/GR	-frtti	C++ RTTI を有効にする
/I	-I	パスを追加する
/Od	-O0	最適化なし
/O1	-O1	サイズを最適化する
/Os	-Os	サイズを最適化する
/Ot	-O2	速度を最適化する
/O2	-O2	速度を最適化する
/TP	-xc++	ファイルを C++として扱う
/u	-undef	全ての定義済みマクロの定義を解除する
/W0 or /w	-w	警告を無効にする
/W1, 2, 3		警告を無視する
/W4	-Wall	最大警告
/WX	-Werror	エラー警告
/X	-nostdinc	標準の収録を無視する

この他の GNU スイッチは、**プロジェクト→設定**で表示される **[C/C++]** ボックスに、前に「-」をつけて入力すれば、どれでも実行できます。たとえば、アセンブラ出力を一覧表示する場合は、このオプションに「`-Wa,al`」を入力します。

-fno-exceptions (例外操作を無効にする) という GNU オプションは、C++ ファイルにデフォルト設定されています。C++ファイルに対する例外操作を有効にしたい場合は、**プロジェクト→ 設定→C/C++→C++言語** のチェックボックスをオンにするか、プロ ジェクトセッティングボックスに/GX と入力します。

リンカオプションも、**プロジェクト→設定**で表示される**リンク**に入力すれば、同様に追加できます。

ELF ファイルを起動/実行する

下の **ELF をロード**ツールバーボタンをクリックすれば、ELF ファイルを起動/実行できます。

ngcrun コマンドラインスイッチのデフォルトは–r です。VSI コントロールペインから、あるいは sn.ini ファイル内に[ngcrun]セクションを作成すると(6ページの「sn.ini - SN System の設定ファイル」参照)オーバーライドされます。

```
[ngcrun]
defaulttarget = ddh0
dvdroot = musyx
args = -p
switches = -v
```

ProDG デバッガとの統合

Visual Studio でプロジェクトを正しくビルトできたら、ProDG デバッガを起動できます。下のような **NGC 用 ProDG デバッガの実行**ツールボタンをクリックします。

アクティブプロジェクト設定は、ProDG デバッガの起動に影響します。プロジェクトがビルドされていて、Nintendo GameCube ビルド設定でアクティブプロジェクト設定として設定 されており、かつ ELF ファイルが適切にビルドされている時は **NGC 用 ProDG デバッガを実行**ボタンをクリックすると、Nintendo GameCube 用 ProDG デバッガが起動し ます。

The **Run ProDG Debugger** 機能は、ターゲットと接続し DVD エミュレータ・ルー トを設定するために VSI Configuration ダイアログにある **NGCRun** オプションを共有します。

ProDG デバッガおよび Target Manager は以下のコマンドラインスイッチで 自動的に起動されます。

```
ngcdbg –vs
```

-vs スイッチは、Visual Studio の互換機能を有効にします。

ngcdbg コマンドラインスイッチは、以下の例に従って sn.ini ファイルの[ngcdbg]セクション(6ページの「sn.ini - SN System の設定ファイル」参照)を作成すればオーバー ライドされます。

```
[ngcdbg]
switches=-vs
```

エクストラスイッチはデフォルト値に追加されます。

プロジェクトにブレークポイントを設定する

Visual Studio 統合機能から呼び出しがあると、すでに Visual Studio のソースで設定されている、すべてのブレークポイントに ProDG デバッガによって自動的にインポート されます。

ProDG デバッガの動作中は、Visual Studio でブレークポイントの設定や変更 を行なっても、ELF ファイルが Visual Studio で再ビルドされたり再ダウンロードされた場合でも、

ProDG デバッガのブレークポイントには影響ありません。一方、ProDG デバッガで設定したブレークポイントは、直ちに Visual Studio に反映されます。このため、ブレー クポイント の設定や変更は、ProDG デバッガのソース、または逆アセンブリペインでのみ行なうことをお勧めします。ここで設定したブレークポイントは、Visual Studio で閲覧できるソースファイルで、自動的に更新されます。

ProDG デバッガを終了すると、その時点でまだ設定されている全てのブレークポイントは、再度 Visual Studio にエクスポートされます。

Visual Studio でソースを編集する

ProDG デバッガでプロジェクトをデバッグしている間はいつでも、ソースペインから Visual Studio エディタに切り換えられます。これを実行するには、ファイルを編集する部分に移動し、ソースペインのショートカットメニューから **Visual Studio で編集**をクリックします。

ソースに加えた変更は、全て新しい ELF ファイルに再ビルドする必要が あります。 これを実行すると、**NGC 用 ProDG デバッガを実行**ツールバーボタンをクリックして Visual Studio から ProDG デバッガを再起動するか、新たにビルドされた ELF ファイルを ProDG デバッガの **ELF をロードして、**ファイルオプションのを使用してダウンロードできます。

ProDG デバッガの動作中に、Visual Studio でブレークポイントを設定、または設定解除し ても無効になりますのでご注意下さい(15ページの「プロジェクトにブレークポイントを設定する」を参照)。

テンポラリファイルを削除する

If you build a project using the Visual Studio 統合機能を用いてプロ ジェクト を作成し、コンパイラオプション--save-temps (28ページの「ngccc オプション」を参照) を指定していると、そのプロジェクトのソースファイルディレクトリに大量のテンポラリファイルが残ります。その状態で makefile を用いてビルドを試みると、そのたくさんのファイルが make ユーティリティに悪影響を及ぼしてしまいます。Make を使ってビルドを行なう前にソースディレクトリからテンポラリファイル(*.s や*.i など)を削除してください。

依存情報の生成

Visual Studio 統合パッケージは依存情報の生成機能をもっています。本セクショ ンでは正しい依存情報を得るために必要な変更について説明します。

Visual C++ はどのようにして依存情報を生成するか

Visual C++がソースファイルの依存関係を生成しそれを維持する手法は、C++のプロジェクトオプション中の/FD オプションで指定できます (**Project > Settings > C/C++ > Project Options**)。/FD が指定されていない場合には、Visual C++ IDE それ自身によって簡易な依存関係が生成されます。これはプロジェクトのソースファイル群をざっと見るだけのシンプルなもので、単に'#include'ディレクティブを探し、それに基づいて依存関係グラフを作ります。ところがこれは、'#include'のコンテクストを見て判断

するわけではないので（文字列やコメントの中からも発見してしまいます）、作り出される依存情報は不完全で間違いを含む可能性があります。

/FD オプションが指定（新しい Visual Studio 統合プロジェクトのデフォルト設定）されると、依存情報は Win32 プリプロセッサパスによって生成されます。この Win32 プリプロセッサは Visual Studio 統合コントロールパネル中の"Skip VC Pass"オプションによって制御 されます。/FD オプションがプロジェクトオプション中で指定されている時は、"Skip VC Pass"オプションが選択されていない場合だけ依存情報が生成されます。 このオプション を選択すると、依存情報を生成しない代わりにビルドが多少速くなります。

頻繁には発生しませんが、Win32 プリプロセッサパスがエラーになった場合には、そのコンパイルモジュールの依存情報は作られません。デフォルトでは Win32 プリプロセッサパスが生成するエラーは表示されません。sn.ini ファイルの'[VSI]'セクションに"showvcerrors=1"というエントリを入力しておくと、ビルド出力ウィンドウにエラーが表示されるようになり、その対処ができるようになります。

ソースのブラウジング

Visual Studio 統合機能は Visual C++が内蔵するソースブラウジング 機能を利 用します。しかし、Visual C++にブラウズするデータを最大限生成させるには変更とワークア ラウンドが必要です。

Visual C++ がブラウズ情報を生成する仕組み

Visual C++のブラウズデータ生成機能は、各コンパイルモジュールの ブラウズデー タを生成するのに Win32 コンパイラを利用します。'/FR'というコマンドパラメータ付で呼ばれると、Win32 コンパイラは出力 SBR (.sbr)ファイルを生成します。ビルド処理の最後に、Win32 は個々の SBR ファイルを集め、ひとつの.bsc ブラウズデータベースにするために BSCMAKE を呼び出します。

ソースは Win32 コンパイラと互換性をもっていなければならないので、ブラウズ情報を生成している時に問題が起こるときがあります。Win32 コンパイラがエラーを発見すると、そのコンパイルモジュールに対する SBR ファイルは生成されないため、ブラウズ情報が不完全になってしまいます。プリプロセッサパスの場合と同様に、エラーがあったコンパイルモジュールの依存情報は生成されません。SDK ヘッダーファイル中の構造とユーザコードが GNU のコンパイル処理としては不正なところは一切なくとも、Win32 のコンパイル中にエラーになることがあります。

NGC_in_VC.h ファイルを用いてブラウズ情報の生成機能を向上させる

Visual Studio 統合機能は、ブラウズ情報と依存情報が正しく生成されるためにソー スが Win32 コンパイラにとって文法的に正しいことだけを必要とします。そのため、Visual Studio 統合機能は特別なヘッダーファイルを強制的にインクルードして Win32 コンパイラを呼び出します。このファイルをカスタマイズして SDK ヘッダーで起こる大半の問題を解決することができます。そのファイル（NGC_in_VC.h）は、Visual Studio 統合プロジェ クトを作った際にアプリケーションウィザードによって自動的に生成されます。NGC_in_VC.h ファイルは統合機能によって自動的に取り扱われるため GameCube プロジェクトでは決して NGC_in_VC.h ファイルを#include してはいけません。NGC_in_VC.h ファイルが SDK のシンタックスエラーの大半を解決するためにプリプロセッサを使用している間、

Win32 コンパイラが文法的に正しいと判断するようにユーザコードを修正する必要が出てくる場合があります。

Win32 コンパイルパス中のエラーによりコンパイルモジュールに対する SBR ファイルが作られない場合、最も簡単な解決法は NGC_in_VC.h ファイルに独自の再定義を追加することです。よく使われるテクニックは、問題の構造をヌルに#define してしまうことです。必要であれば、もっと手の込んだプロプロセッサマクロを使うこともあります。

ソースコードを修正してブラウズ情報の生成を向上させる

NGC_in_VC.h ファイルを修正しても Win32 コンパイルパス中で文法エラーがなくならない場合（複数行の asm ステートメントなど）、ソースコードを修正しなければならない時もあります。_WIN32 および SN_TARGET_NGC は共にコンパイルパス中にのみ定義されるため、NGC ソースコードの修正としては簡単なケースで次のようになります。

変更前:

```
asm("
...
...
");
```

変更後:

```
#ifdef SN_TARGET_NGC
#ifndef _WIN32
asm("
...
...
");
#endif
#endif
```

これで問題のコードは Visual Studio 統合プロジェクトの Win32 コンパイラパスでコ ンパイルされなくなります。

makefile をソースブラウジングに使う

プロジェクトをビルドする際に外部の makefile を使用している場合は、それからブラウズ情報を生成することもできます。Visual C++ヘルプライブラリに Visual C++ IDE の外の ブラウズ情報を生成する方法が Visual C++ヘルプライブラリの Q102326 に記述され ています。

トラブルシューティング

サードパーティ統合ブレークのインストール

Visual Studio 統合機能 をインストールしたあとに、既存の Visual Studio ビルド設定 (例: Microsoft XBox SDK) が正常に機能しなくなることがあります。

標準 Microsoft Visual C++ ビルドツール (cl.exe, link.exe, lib.exe) は、C:\Program Files\Microsoft Visual Studio\VC98\Bin ディレクトリにデフォルトで設定されています。ProDG Visual Studio 統合機能を インストールすると、既存のラッパーツール (sncl.exe, snlink.exe, snlib.exe) もまた、同じディレクトリに設定され、以下のレジストリー キーが修正されて、 Microsoft のツールに代わって起動します:

```
HKEY_CURRENT_USER\Software\Microsoft\DevStudio\6.0\Build
System\Components\Platforms\Win32 (x86)\Tools\32-bit C/C++
Compiler for 80x86\Executable Path=sncl.exe

HKEY_CURRENT_USER\Software\Microsoft\DevStudio\6.0\Build
System\Components\Platforms\Win32 (x86)\Tools\COFF Linker for
80x86\Executable Path=snLink.exe

HKEY_CURRENT_USER\Software\Microsoft\DevStudio\6.0\Build
System\Components\Tools\Library Manager\Executable
Path=snlib.exe
```

ただし、XBox SDK をインストールすると、これらのレジストリー キーは元のように変更 されます:

```
HKEY_CURRENT_USER\Software\Microsoft\DevStudio\6.0\Build
System\Components\Platforms\Win32 (x86)\Tools\32-bit C/C++
Compiler for 80x86\Executable Path=cl.exe

HKEY_CURRENT_USER\Software\Microsoft\DevStudio\6.0\Build
System\Components\Platforms\Win32 (x86)\Tools\COFF Linker for
80x86\Executable Path=link.exe

HKEY_CURRENT_USER\Software\Microsoft\DevStudio\6.0\Build
System\Components\Tools\Library Manager\Executable
Path=link.exe /lib
```

通常、XBox SDK は、C:\Program Files\Microsoft XBOX SDK\VC7 ディレクトリにエグゼクタブル (cl.exe, link.exe, lib.exe とも呼ばれる) を設定しています。問題は、Visual Studio 統合機能 設定が、Nintendo GameCube プロジェクトでない ビルドを探知すると、すべてのコマンドをデフォルトのエグゼクタブルにパスしてしまうことです。そこで、XBox プロジェクトが間違ったファイルでビルドされると、(C:\Program Files \ Microsoft XBOX SDK\VC7 の XBox ではなく、C:\Program Files\Microsoft Visual Studio\VC98\Bin のデフォルトの Win32 ファイル) が呼び出されます。

この問題を解決するには 3 つの方法があります:

1. C:\Program Files\Microsoft Visual Studio\VC98\Bin ディレクトリの cl.exe, link.exe と lib.exe プログラム を削除/名前の 変更をし、XBox バージョンのパスを **オプション > ディレクトリ > エグゼクタブル ファイル** に追加します。

2. sncl.exe, snlink.exe と snlib.exe プログラムを C:\Program Files\ Microsoft XBOX SDK\VC7 ディレクトリに移動し、このディレク トリが **オプション> ディレクトリ > エグゼクタブル ファイル**で指定されていること を確認します。

3. sncl.exe, snlink.exe と snlib.exe プログラムを独自の専用ディレクトリに 移動します。このディレクトリを**オプション > ディレクトリ > エグゼクタブル ファイル** に追加します。C:\Program Files\Microsoft Visual Studio \VC98\Bin と C:\Program Files\Microsoft XBOX SDK\VC7 ディレクトリの両方が、**オプション > ディレクトリ > エグゼクタブル ファイル** で指定されていることを確認します。Visual Studio 統合機能 **エグゼクタブル** によって、リスト内の最高位のディレクトリから Microsoft **エグゼクタブル** が確実に実行されます。

「未定義シンボルへ参照」エラー

ngcld リンカは、コマンドラインで指定されたライブラリの順序に依存しています。ライブラリがコマンドラインでそれ以前に指定された他のライブラリのシンボルを参照すると、「未定義シンボルへ参照」エラーが表示されます。

Visual Studio 統合機能 コントロールパネルの [グループ ライブラリ] オプションを 選択すると、この命令依存は削除されます。 ライブラリをグループ化すると、ビルド時間が 延長されるので注意してください。この問題を解決するもっとも効果的な方法は、ライブラリがコマンドラインでそれ以前にリストされた他のライブラリのシンボルを参照しないように、ライブラリに命令することです。

第 4 章: C と C++でのメモリ管理

序文

高度な柔軟性とリソースアクセスを発揮するためには、任天堂 SDK のアーキテクチャにおいてアプリケーションプログラマは作成したプログラムに適切なメモリ管理ルーチンを設定する必要があります。どんな種類のメモリ管理要求にも合うようなデフォルトの実装である通常の C や C++のメモリ管理ルーチンではそれは不可能です。

ProDG ライブラリは可能な限り動的なメモリ割り当てを使わないように設計されています。しかし、いくつかのライブラリ機能(例えば STL IOStreams、strdup 等)ではメ モリ管理が必須です。これらの機能を利用する場合、ライブラリがヒープメモリにアクセスするために最適なユーザ実装のメモリ管理ルーチンが登録されていなければなりません。ランタイム実装なしにどの標準メモリ管理ルーチン(`malloc/new`, `free/delete`)を呼び出し ても結果は致命的なランタイムエラーになります。プログラムは中止され、次のテキスト文がデバッグ TTY ストリームに表示されます。

malloc/free に C++例外操作が抜けている場合

```
*** Library error ***
Exception handling requires access to heap memory
Hook _register_malloc to your malloc routine - see ProDG manual.
*** Library error ***
Exception handling requires access to heap memory
Hook _register_free to your heap freeing routine - see ProDG manual.
```

strdup (malloc のみ)

```
*** Library error ***
The function 'strdup' has attempted to allocate heap memory
without a definition of _register_malloc  - see ProDG manual.
```

setvbuf (malloc/free)

```
*** Library error ***
The function 'setvbuf' has attempted to allocate heap memory
without a definition of _register_malloc - see ProDG manual.
*** Library error ***
The function 'setvbuf' has attempted to free heap memory
without a definition of _register_free - see ProDG manual.
```

malloc を直接呼出し

```
*** Library error ***
An external call has been made to 'malloc'
Calling function address 0x8000350C
```

```
Please See the ProDG manual
```

free を直接呼出し

```
*** Library error ***
An external call has been made to 'free'
Calling function address 0x8000350C
Please See the ProDG manual
```

malloc と free を使ったプログラミング

ProDG に付属して出荷される C ライブラリはユーザが提供するメモリ管理ルーチンにアクセスするために登録技術を使用します。デフォルトではルーチンはなにも登録されていないので、動的なメモリ割り当てを必要とするライブラリルーチンを使用するとランタイムエラーになってしまいます。

登録モジュールへのインタフェースは malloc.h または stdlib.h ファイルにあります。次の 例のようにルーチンに登録関数ポインタを設定して自分用のハンドラを登録します。

```
void* MyMalloc ( size_t size )
{
  // your malloc code here
  return ptr ;
} ;

void MyFree ( void* mem )
{
  // your free code here
} ;

int main ()
{
_register_free = MyFree ;
_register_malloc = MyMalloc ;
 // ....
} ;
```

malloc と free ルーチンの実際の実装はユーザコードで提供されています。これらのルーチンでは、メモリを割り当てたり解放するシステムライブラリの要求がメインユーザコード、SDK、サードパーティのライブラリと干渉しないようにしなければいけません。

最も簡単な実装は単に SDK ヒープルーチンにフックをつける方法です。

以下は任天堂のヒープ機能を利用した `malloc` と `free` の実装例です。

```
void* malloc (size_t blocksize)
{
    //IsHeapInitialized is a global bool
    if (!IsHeapInitialized)   {
        HeapInit();
    }
    return OSAllocFromHeap(HEAP_ID, blocksize);
}

void free (void* ptr)
```

```
{
    OSFreeToHeap(HEAP_ID, ptr);
}
```

HeapInit() がヒープを初期化し、サイズと配列を設定します。

```
static void HeapInit()
{
    void*    arenaLo;
    void*    arenaHi;

    if (IsHeapInitialized)
    {
        return;
    }
    arenaLo = OSGetArenaLo();
    arenaHi = OSGetArenaHi();

    // Create a heap
    // OSInitAlloc should only ever be invoked once.
    arenaLo = OSInitAlloc(arenaLo, arenaHi, 1); // 1 heap
    OSSetArenaLo(arenaLo);

    // Ensure boundaries are 32B aligned
    arenaLo = (void*)OSRoundUp32B(arenaLo);
    arenaHi = (void*)OSRoundDown32B(arenaHi);
    // The boundaries given to OSCreateHeap should be 32B aligned
    OSSetCurrentHeap(OSCreateHeap(arenaLo, arenaHi));
    // From here on out, OSAlloc and OSFree behave like
    // malloc and free respectively
    OSSetArenaLo(arenaLo=arenaHi);
    IsHeapInitialized = TRUE;
}
```

new と delete を利用した C++のプログラム

C++プログラムで new と delete を利用する場合にも、C プログラムで malloc と free を利用する場合と同様な注意が必要です。ユーザ定義のルーチンが見つからない場合、標準ライブラリは致命的なランタイムエラーを発生します。C++ではユーザのメソッドでシステムルーチンを書き換えることができるので、ユーザコードを組み入れてリンクするためにこの方法が使われます。

注: プログラムで C++の例外を利用している場合、システムライブラリも上記 で説明した登録技術を介してヒープストレージにアクセスする必要があります。

ProDG インストーラはユーザが提供する new と delete の例を project\ SNDemos\ CPPDemo にインストールします。

I/O ストリームバッファを使用する

現在、fopen, fread 等のバッファ付ファイル入出力関数を使用する場合、これらの 関数はバッファ無シングルキャラクタモードで実装される必要があります。

I/O 速度を向上させるためにはいくつかの方法があります。

1. PCwrite 関数を利用して入出力を手動でラインバッファさせる。
2. OSAllocFromHeap を利用してメモリを割り当て、それを SN I/O バッファ に指定する。
3. OSAllocFromHeap を利用してメモリを割り当て、setvbuf を利用 して現在のストリームに指定する。
4. 自分で malloc と free を定義し、そのストリームで setbuf を使用する。

以下は、それぞれの方法の簡単な例です。

1. PCwrite 関数を利用して入出力を手動でラインバッファさせる:

```
#include <libsn.h>

int k;
int numwritten;
k = PCopen("data.txt", 2, 0);
numwritten = PCwrite( k, "hello all,", 10); //10 is number of chars to
write
numwritten = PCwrite( k, “from PCwrite.”, 13);
PCclose( k );
```

2. OSAllocFromHeap を利用してメモリを割り当て、それを SN I/O バッファに指定する:

```
struct __sbuf *buf = _sn_IO_buf_ptr;
FILE* stream;
char* string_buf;
unsigned char* mybuf;
int numwritten;

(Init Heap)

string_buf = (char*) OSAllocFromHeap(HEAP_ID, 2048);
mybuf = (char*) OSAllocFromHeap(HEAP_ID, 2048);

buf[0]._base = mybuf;
buf[0]._size = 2048;

strcpy (string_buf, "I want to write this string to the HD\n");

if ((stream = fopen ("data.txt", "w"))== NULL)
        printf("\nFile Error");
else
        printf("\nFile opened for write");
```

```
if (stream != NULL)
    {
  numwritten = fwrite( string_buf, strlen(string_buf), 1, stream );
 }
fclose(stream);
```

_sn_IO_buf_ptr は 10 個の__sbuf オブジェクトからなる配列へのポインタです。__sbuf は 2 つの要素で構成されます。_base はメモリエリア(バッファの先頭)へのポインタで、_size はユーザが割り当ててバッファとして使用したいメモリエリアのサイズです。

fopen は呼び出されると利用可能な 10 個のファイルストリームの 1 つを利用します。一番最初に開かれるものを#0 とするので、メモリを_sn_IO_buf_ptr[0]に割り当てるとオープンされる最初のファイルストリームには自動的にバッファが指定されます。そのバッファがクローズされ新しいバッファが開始されると再び#0 が使用されます。

この方法はバッファを 1 度だけ指定すればその後は自動的にバッファリングされるので便利です。

3. OSAllocFromHeap を利用してメモリを割り当て、`setvbuf` を利用して現在のストリームに指定する:

```
mybuf = (char*) OSAllocFromHeap(HEAP_ID, 2048);
if ( (stream = fopen ("data.txt", "w")) == NULL)
        printf("\nFile Error");
else
        printf("\nFile opened for write");
setvbuf (stream, mybuf, _IOFBF, 2048);  //_IOFBF = fully buffered
```

これは標準的なファイルバッファリングの C における実装です。

4. 自分で malloc と free を定義し、そのストリームで setbuf を使用する:

この方法は一度 malloc と free を定義するとヒープメモリを自動的に割り当てる関数を利用できるという長所があります。

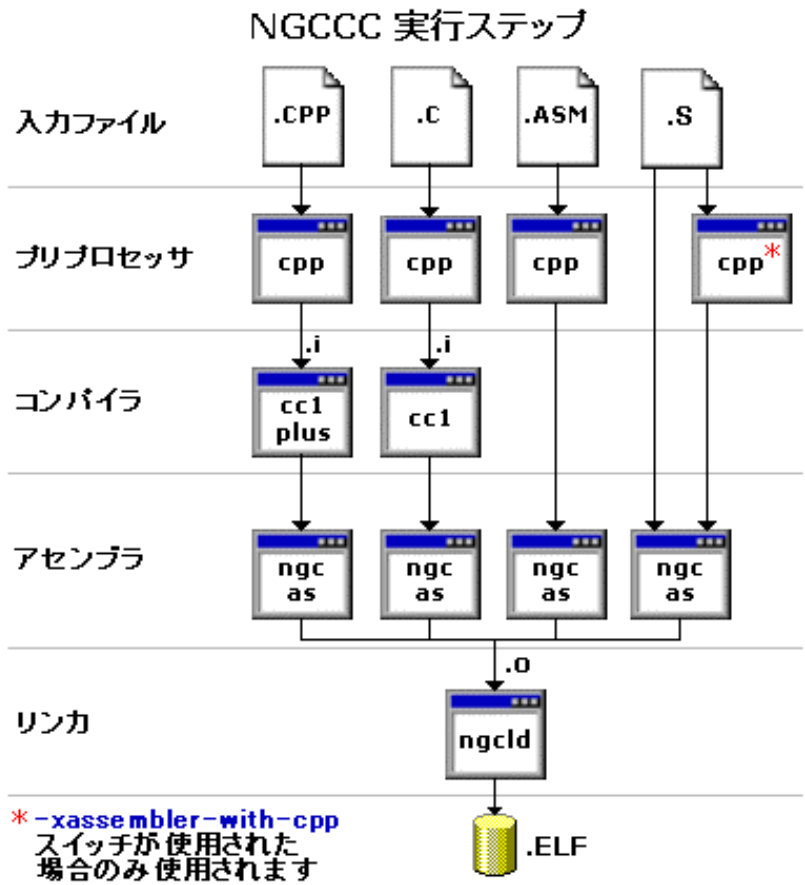
[このページは意図的に空白にされています]

第 5 章:　ProDG コンパイラ ドライバ

ngccc コンパイラ ドライバの使用

ngccc コンパイラ ドライバは、エグゼクタブル コードを生成するために、Nintendo GameCube コンパイラ、アセンブラ、およびリンカへの、ProDG の呼び出しを自動化するツールです。各ソースファイルが、どのコンパイラ、またはアセンブラを使うかを決定するために、コマンドライン スイッチとファイル名エクステンションを使って、そこで結果とし て生じた出力を次のリンカ フェーズへとパスします。これによって、各ツールを個別 に呼び出す必要がなくなります。

以下の図はこれらの異なるステップそれぞれの間の関係を示しています。



コンパイラドライバによって、他のツールに対して自動的に多数のパラメータが生成されます。各ツールに何がパスされているかを正確に表示するには、'-v' (verbose) 出力オプションを指定してください。これによって、各ツールが呼び出されるのに呼応して、すべてのパラメータが標準出力に書き込まれます。

ngccc.exe 実行ファイルは、検索パスのどこに入力しても検索できますが、環境変数 SN_NGC_PATH=...を呼び出せば、プログラムが sn.ini(SN Systems 設定)を検出できま す。詳細は 6 ページの「sn.ini - SN System の設定ファイル」を参照。

プロジェクトを問題なくビルドさせるため、以下の環境変数(ここでは一般的な設定を例示)を sn.ini 設定ファイルの[ngccc]セクションに設定する必要があります。

```
[ngccc]
....
assembler_name=ngcas
opt_assembler_name=ngcas
linker_name=ngcld
linker_path=<install dir>\bin
linker_script=<install dir>\bin\ngc.ld
....
```

再配置可能モジュールをビルドしている場合には次の環境変数を利用する必要があります(標準的な設定を例示します)。

```
rel_link_file=<install dir>\lib\plfkeep.lst
rel_linker_script=<install dir>\bin\plf.ld
```

ここで、<install dir> は、ProDG for Nintendo GameCube がインストールされ ているディレクトリで、C:\Program Files\SN Systems\ngc のようになります。

ngccc コマンドライン書式

以下が、ngccc コンパイラ ドライバ コマンドライン書式です:

```
ngccc [options] filename [filename…]
```

GNU プロジェクトをポートする場合は、メークファイル内で gcc への呼び出しを ngccc へと変更することができますが、 指定したすべてのオプションが ngccc によってサポートされていることを、必ず確認してください。

ngccc オプション

コンパイラ ドライバ オプションはファイル名リストに先行します。以下の表では、さまざま なコンパイラオプションを、タイプ別にグループ分けしています。

処理制御および出力

オプション	アクション
-c	オブジェクトファイルにコンパイルする。出力ファイルが (-o オプションで) 指定されている場合は、全ての出力がその ファイルに送られる。指定されていない場合は、出力ファイルに、拡張子を.o に変更したファイル名をつけられる。
-E	前処理のみ。出力ファイルが指定されていない場合は、出力は画面に表示される。
-o <file>{,<mapfile>}	デフォルトのファイル名を使用せずに、<file>で出力ファイル名を指定する。 マップファイルの作成には、出力 <mapfile>直後に(スペースなし)でファイル名を追加。
-S	アセンブラソースにコンパイルする。出力ファイルが指定されていない場合は、出力ファイルは元の名称に新たな拡張子.S が付いたものとなる。
-save-temps	前処理出力やコンパイラ作成アセンブラソースなど、中間的一時ファイルを保存する。
-v	詳細モード。実行前に全てのコマンドを表示する。
-x<type>	これに続く入力ファイルを、タイプとして扱う。 34 ページの「デフォルトファイル拡張子アクションを最優先する」を参照。

C/C++言語オプション

オプション	アクション
-ansi	ANSI 準拠の確認コード。
-f	コンパイラオプション/最適化を指定する (GNU の Web サイト **www.gnu.org**. でオンライン ドキュメントを参 照)。
-fbyte-bool	デフォルトの 4 バイとではなく、C++ 'bool' タイプ 1 バイトロングにする。**注意**:この変更はプロジェクト全体で使われなければならない (つまり、コンパイラユニット間 でブールのサイズをミックスしてはならない)。
-fdefault-single-float	常数デフォルトを複式ではなく単式として使用、"f"の指定を不要とする。
-fforce-link-once	デバッグデータが生成されている場合であっても、テンプレート インスタンスとインプライド インライン メソッドがリンクワ ンスセクションで設置されるよう強制する (デフォルトでは、デバッグ中にリンクワンスセクションを使用しない)。これによって、リンク時のテンプレートインスタンスとインプライド インライン メソッドの反復が自動的に削除される(下記参 照)。
-fno-exceptions	C++ 言語例外機能を無効にする。プログラムが C++ 例外を使っていない場合でも、C++ 例外機能によって、大量の特

	別コードとデータを生成させられる。
-fno-implement-inlines	インライン機能の非インラインインスタンスを生成しない、詳細は 67 ページの「重複する非インライン関数の生成防止」を参照。
-fno-rtti	C++ の RTTI (ランタイム タイプ情報)を処理するコー ドを生成しません。
-fno-static-dtors	スタティック グローバル クラス インスタンスのデストラクタ コードの生成を抑制。詳細は 67 ページの「グローバル デストラクタの削除」を参照。
-fopt-dwarf	DWARF デバッグ情報最適化をオンにする
-fshort-wchar	wchar の長さを強制的に 2 バイトにする (標準ライブラ リ関数を使用する場合、代わりの libc にリンクする必要がある)。
-fvtable-thunks	仮想ディスパッチテーブル('vtable')を実現するために 'thunks'を使います。

非デバッグ C++ビルドでは、コンパイラは'link_once'セクションにテンプレートインスタンスを生成します。これらのセクションは、リンカがそれぞれのテンプレートインスタンスの1つのコピーが、最終段階のプログラムにリンクされる事を確認する為使用されます。デバッグビルドでは、コンパイラはデフォルトでフルデバッギングが出来るよう複数のテンプレートインスタンス用の' weak' シンボルを使用します(これは、最終のコード・サイズを膨らます)。デバッグビルドでも、link_once セクションを生成する為にコンパイラに-fforce-link-once スウィッチを使用してください。ただし、テンプレートコードのフルデバッギングは 出来ないかもしれません。

警告オプション

オプション	アクション
-W...	個別の警告を有効にする(または、 -Wno-...を使っ て無効にする) (GNU の Web サイト**www.gnu.org**.でコンパイラドキュメントを参照)。
-W	–Wall で有効にされなかった特別警告を有効にする。46 ページの「警告を発しない C コンパイラ」を参照。
-Wall	全ての警告を有効にする。
-Wno-missing-enum-case	計数を基準にした変更の欠落ケースへの警告を生成しない。
-Wpromote-doubles	フロートタイプが倍化する増進の生成を警告する。 #pragma mathwarn を使用して関数上で設定 する事も可能 (詳細は、以下参照)。
-Wsoftware-extern-library	ソフトウェア マス ライブラリ ルーチンへの呼び出しの生成を警告する。

-Wsoftware-math-library	ソフトウェア マス ランタイム ルーチンへの呼び出しの 生成を警告する。
-w	全ての警告を無効にする。

-Wpromote-doubles 警告の設定は、関数上で#pragma mathwarn を使用する事により可能です。

使用法:

```
#pragma mathwarn on
#pragma mathwarn off
```

on に設定された場合、関数はコンパイラがフロートからダブルに設定したり、ソフトウェアフローティング ポイント マス ルーチンを呼び出した時に警告します。

デバッグオプション

オプション	アクション
-g	ソースレベルでのデバッッギングのデバッグ情報を生成する。 **注意**: デバッガを使用する場合は、このオプションが必須。

最適化オプション

オプション	アクション
-G<size>	GP レジスタの最適化のための変数 <size> を設定する: 0=none.
-O0	最適化なし (デフォルト)。
-O or -O1	標準レベルの最適化。
-O2	完全最適化。
-O3	完全最適化、関数直列化 。
-Os	コードをできる限り小さく最適化する。

前処理機能オプション

オプション	アクション
-I<dir>	収録ファイルの検索で検索対象となるディレクトリのリストにこのパスを追加する。
-D<name>	前処理機能のシンボル名を<name>で定義する。
-D<name>=<def>	<def>を値とする前処理機能のシンボル名を<name>で定義する。
-U<name>	<name>で定義されたシンボル名を前処理実行前に解除する。

-Wp,...	前処理機能のオプションを指定する (GNU の Web サイト **www.gnu.org**. でオンライン ドキュメントを参照)。
-M	標準出力に依存情報を生成する。前処理のみ実行され、ファイルはコンパイルされない。
-MM	-M とほぼ同じ。違いは、引用符で囲まれた収録ファイルが. #include "file.h"と依存ファイルに追加されること。
-MD	通常通りにコンパイルされると同時に依存ファイルが生成される。
-MMD	システムファイルが除かれる以外は-MD と同じ。
-MG	-M と同様だが、見つからないファイルはビルドの最中 に現行ディレクトリ中で自動的に生成されるものとみなす。

アセンブラ オプション

オプション	アクション
-Wa,...	アセンブラにオプションを指定する。
-asmpp	ngcasm C 'asm'互換フィルタを呼び出す。
-noasmpp	ngcasm C 'asm'互換フィルタを呼び出さない (デフォルト)。

アセンブラには、必ずコンマを使って別々に区分してください。多くのコンパイラのコマンドライン オプションが、自動的にコンパイラを使ってアセンブラにパスするの で、通常は、この方法を使う必要はありません。

リンカ オプション

オプション	アクション
-keep<file>	sn.ini ファイル内 rel_link_file=で指定されたファ イルより優先して<file>内のすべてのシンボルを維持する (6ページの「sn.ini - SN System の設定ファイル」を参 照)。 複数の'keep symbols'ファイルを指定する時は-Wl、-keep、plfkeep2.lst、-keep、plfkeep3.lst 等を利用 する。この場合 rel_link_file= で指定されたファ イルも使用される。
-l<library>	リンク時に特定ライブラリ <library> が含まれる。
-L<dir>	ライブラリの検索時に検索対象となるディレクトリのリストにこのパスを追加する。
-X... or –Wl,...	リンカに引き渡されるオプションを指定する。
-nostdlib	sn.ini に一覧表示されている標準のライブラリをリンクに組み込まない。
-linkscript file	指定されたファイルをリンカスクリプトとして使用する。

-TR <file>	sn.ini ファイルの linker_script=で指定されたファイル より優先して<file>を再配置可能リンカスクリプトとして使用する。

機種依存オプション

オプション	アクション
-m...	機種依存コンパイラオプションを指定する(GNU の Web サイト **www.gnu.org**. でコンパイラ ドキュメント を参照)。

ファイル名

ngccc は、以下のどのタイプのファイルも入力として受け入れることが可能で、ファイル名の拡張子に従って以下のアクションを適用します:

ファイルタイプ	拡張子	アクション
C ソース	.C	プリプロセス、コンパイル、アセンブル、リンク
C++ ソース	.CC, .CPP	プリプロセス、コンパイル、アセンブル、リンク
前処理 C ソース	.I	コンパイル、アセンブル、リンク
前処理 C++ ソース	.II, .IPP	コンパイル、アセンブル、リンク
コンパイラ ソース アセンブラ	.S	アセンブル、リンク
ユーザー ソース アセンブラ	.ASM	プリプロセス、アセンブル
C ヘッダ	.H, .HPP	なし
事前に部分的にリンクされた ELF	.PREPLF	リンクして再配置可能モジュールを作成する(71ページの「再配置可能モジュールをビルドする」を参照)。.
オブジェクトファイル	その他全て	リンクのみ

- 特定のファイルタイプを指定していないと認識された拡張子を持つファイルは、オブジェクトファイルとして扱われ、リンカのみにパスされます。これには、標準オブジェクトファイル拡張子の.o files が含まれます。

- 使われる拡張子の数に制限はありません; ngccc は、一度の 呼び出しで、多数の C と C++ ファイルをコンパイルでき、それぞれに適切なコ ンパイラ を適用します。

取られるアクションは、自動的にリンキングを削除する –c のようなコントロールオ プションの対象となります。

具体例:

```
ngccc -c -O2 main.c objects.c pluscode.cpp
```

このコマンドは、main.c、objects.c、pluscode.cpp を前処理、コンパイル、アセンブル、最適化、かつデバッグ情報を含まずにコンパイルされた 3 つのオブジェクトファイルを作成します。main.c ファイル、および objects.c ファイルは、C コンパイラでコンパイルされます。一方、pluscode.cpp は C++コンパイラでコンパイルされます。

```
ngccc -c *.c -I. -Ic:\include
```

このコマンドは、現在のディレクトリにある全ての.c ファイルを前処理、コンパイル、アセンブルして、.o ファイルを作成します。#include <…> で入力されたファイルは、まず現在のディレクトリ内で検索され、次に c:\include で検索されます。

```
ngccc -g -O2 *.c -o main.elf
```

このコマンドは、すべての.c ファイルを前処理、コンパイルし、一組のオブジェクトファイル *.o を作成します。その後、これらのオブジェクトファイルは関連付けされ、実行ファイル main.elf をビルドします。コンパイラの最適化が有効となり、出力ファイルにはデバッグ情報が添付されます。

デフォルトファイル拡張子アクションを最優先する

-x<type> オプションを使ってコマンドラインの後続ファイルを、通常ファイル拡張子に よって表示されているタイプとして優先させ、譲渡タイプに指定する事も可能です。また、複数の-x… オプションも指定ができ、次の -x… オプションが出現するまで、 すべての後続 ファイルに影響を与えます。

<type>	**、、、と見なされるファイル**
c	C ソース
cpp-output	前処理 C ソース
c++	C++ ソース
c++-cpp-output	前処理 C++ ソース
assembler	アセンブラ
assembler-with-cpp	アセンブラ
c-header	C ヘッダ
<none>	オブジェクト

ゲームキューブライブラリを指定する

任天堂のライブラリを使ってビルドを実行するには、以下の文字列を使用します。

```
ngccc ... $(OBJS) ... --start-group -lsn
$(OSLIBS) -lc -lgcc -lm --end-group
```

ここで、「$(OSLIBS)」は、要求されているニンテンドーゲームキューブのライブラリです。

応答ファイルを使用する

応答ファイルは、長いコマンドラインの繰り返し入力作業の省略に使用できます。応答ファイルを作成するには、オプションをスペース、タブ、改行などで区切って、ASCII テキストファイルに入力します。ngccc をコマンドラインで呼び出す場合は、前に「コマーシャルアット(@)」文字をつけてファイル名を入力すれば、応答ファイルの使用を指定できます。入力例は以下の通りです。

```
ngccc @myresponsefile
```

ngccc は、「myresponsefile」の内容を使用して、引数を取得します。

C プログラミングに関する考慮事項

このセクションでは、ニンテンドーゲームキューブ用プログラムを開発する際に比較的重要となる、C プログラミングに関するいくつかの考察事項について解説します。

グローバル変数のメモリ場所の指定

GNU 属性機能の拡張機能として、グローバル変数の場所を、一定のアドレスに指定できるというものがあります。たとえば、以下のように入力するとします。

```
int example __attribute__ ((address(0x1234))) ;
```

この場合、正数変数 example は、ゲームキューブメモリマップの 0x1234 というアドレスに指定されています。

新 sync イントリンシック

新たなイントリンシック関数「__sync」 によって Gekko CPU の sync オプコードにアクセ スできます。これは、CPU パイプライン内のすべてのオプコードを sync オプコードの前に完了するものです。さらに sync が完了するまで、新たなオプコードが発信されることはありません。またこの関数によって、以下のように最適化が進行中に、コードストリーム内で sync の正しい位置を確認できます。

```
 __sync() ;   // synchronize the CPU
```

Naked 関数のサポート

コンパイラは、通常の C 入力や関数終了コードの削除に対応しています。この機能は、主にユーザが自分でアセンブラベースのルーチンを記述する際に使用されます。書式は以下の通りです。

```
__attribute__ ((naked))
```

これは、関数宣言文ではなく、関数プロトタイプに追加すべきものであることに注意して下さい。たとえば以下のようになります。

```
void MyNakedFunc () __attribute__ ((naked)) ;

void MyNakedFunc ()
```

```
{

// exit コードも entry コードも作成されませんでした

} ;
```

高速キャスティング

ゲームキューブに搭載された Gekko CPU には、ハードウェアで実行される単精度フロートと小さい整数のキャスティングを可能にするという特徴があります。 NGC SDK では、ライブラリモジュールを通じてこの特徴にアクセスできます。このコンパイラ変更によって、コンパイラは自動的に高速キャスティングコードを生成します。これは、コードの最適化などに関する限り、ライブラリメソッドより多くの利点を有します。

キャスティングを実行するために、CPU はグラフィックス定量化レジスタ(GQR)を使用します。高速キャスティングによってより小型かつ高速のコードが実現されますが、使用前にプログラムで 4 つの GQR を設定しておく必要があります。

実行

コンパイラ上での高速キャスティングコードの使用は任意選択であるため、コンパイラには機種固有のスイッチ'-mfast-cast' と高速キャスティングに適合する'-mno-fast-cast' があります。デフォルトでは、高速キャスティングモードが無効になっています。高速キャスティングモードが、コンパイラで無効になっている時は、ソフトウェアで浮動小数点値との変換を行います。

高速キャスティングが有効の時は、コンパイラが 4 つの GQRs を用いて変換を行ないます。サポートしている操作は以下の通りです。

- 符号つきあるいは符号なしバイト からフロートへの変換
- 符号つきあるいは符号なしショート int (16-bits)からフロートへの変換
- フロートから符号つきあるいは符号なしバイトへの変換
- フロートから符号つきあるいは符号なしショート int (16-bits)への変換

コンパイラは、以下のように、標準 C 書式から自動的に関連する操作を検出します。

```
short int si1 ;  // 16 bits
int i1 ;         // 32bits (高速キャスト不向き)
float f1 ;

f1 = si1 ;   // 高速キャストを実行
f1 = i1 ;    // 通常のソフトウェアキャストを実行
```

プログラミングにおける考慮事項

高速キャスティング操作では、CPU のシングルペアモードを使用しますが、高速モードを使用するためにコンパイラの PS モード生成(-mps-float)をオンにする必要はありません。

デフォルトの高速キャスティングモードでは、コンパイラは、以下の GQR レジスタにあらかじめ設定された値が含まれているとみなします。これらの GRQ レジスタはユーザコードで変更しないで下さい。これらの定義は、任天堂 SDK OSFastCast ライブラリモジュールで使 用される定義に一致するものです。

GQR レジスタ	使用法
2	符号なしバイト変換
3	符号なし 16-bit 変換
4	符号つきバイト変換
5	符号つき 16-bit 変換

高速モードを使用するためにモジュールをコンパイルする場合は、プログラムの最初にこれらのレジスタをあらかじめ設定する必要があります。簡単に設定を行なうには、以下のように SDK OSFastCast.h ライブラリから「OSInitFastCast」ルーチンを呼び出します。

```
#include <OSFastCast.h>
int main ()
{
  OSInitFastCast() ;  // GQR あり
  .
  .
  return 0 ;
} ;
```

単一ペア浮動小数点演算

ゲームキューブに搭載された Gekko CPU の特徴に、単一ペア浮動小数点値を単一オ プコードで計算できることがあります。これにより、GNU コンパイラは Gekko CPU の シングルペアモードをサポートできます。これはベクトルおよびその他の行列形式演算の高速化に役立ちます。

単一ペア(PS)サポートはネイティブ C/C++ サポートと SNAPS(SN API for Paired Singles)の 2 つの方法で提供されます。ネイティブモードでは通常の C/C++ 言語がもつ 機能で簡単に PS モードにアクセスできます。SNAPS にはアセンブラコードに頼らずに PS オプコードに直接アクセスできる一連のビルトインコンパイラ関数があります。SN PS がア センブラよりも優れている点は、PS オプコードがコンパイラのバックエンドで完全に表現されていることです。これにより PS オプコードは他の CPU 機能と同様に扱われ、数々の最適化パスで通常コードと見なされます。

タイプの実装

コンパイラは、GNU コンパイラの「__attribute__」機能を介してアクセスした新たなマシンモードを実行します。この新しいモードはシングルペアモード「PS」といいます。あるいは GNU の一般的なモード名である「V2SF」(2 つのシングルフロートの意味)が使われます。

以下の C++コードは PS モードの 1 タイプを宣言しています。

```
typedef float vector __attribute__ ((mode (PS)));

typedef float vector __attribute__ ((mode (V2SF)));
```

定義内に宣言されている C++ タイプは重要ではありませんが、通常の C++操作で使用しない方法として「void」として宣言すると良いでしょう。

ゲームキューブにおけるシングルペアモードの使用は任意選択であるため、コンパイラには、機種固有スイッチ「-mps-float」と、シングルペアモードに適合する「-mno-ps-float」があります。デフォルトでは PS モードは無効になっています。PS モードがコンパイラで無効の時に、PS モード変数を使用するとコンパイルエラーとなります。

さらに、「aligned」属性を介して PS モードタイプのデフォルトの配置を書き換えることができます。

例えば、PS モードを宣言する次の 2 つの C++コードの配置は違ったものになります。

```
typedef float vector1 __attribute__ ((mode(PS),aligned(2)));

typedef float vector2 __attribute__ ((mode(PS),aligned(4)));
```

ユーザコードで様々なモードの使用を簡素化するために、コンパイラは PS モードが有効の時にビルトインタイプを定義します。そのレイアウトは以下の通りです。

```
typedef union SN_PS_type
{
  void ps __attribute__ ((MODE (PS))) ;
  struct tflt
  {
    float hi, lo ;
  };
  tflt flt ;
} ;
```

変換

PS タイプと、その他のデータタイプとの自動変換はサポートされていません。PS タイプのインスタンスに最も簡単にアクセスする方法は、ビルトインユニオンにアクセスすることです。

```
SN_PS_type vec
vec.flt.lo = 1.0f ;
```

これによって、以下のように各 PS 演算数に含まれる 2 つの単精度浮動小数点値にアクセスできます。

```
void inline SetVector (SN_PS_type& v, float x, float y)
{
  v flt.hi = x ;
  v.flt.lo = y ;
} ;

void inline GetVector (float& x, float& y, SN_PS_type v)
{
  x = v.flt.hi ;
  y = v.flt.lo ;
} ;
```

ネイティブ PS 演算

PS 変数にネイティブモードでアクセスするためには標準的な C/C++演算子を使用します。PS モード変数にサポートされる演算は以下の通りです。

演算	書式	Gekko オプコード
足し算	+	ps_add
引き算	-	ps_sub
掛け算	*	ps_mul
割り算	/	ps_div
ネゲート	-	ps_neg

コンパイラは通常通り、浮動小数点レジスタに対する演算数の配置および動きを操作します。すなわち、以下のように PS タイプの変数を宣言し、そこで通常の C++ 演算を行ないま す。

```
SN_PS_type qaz1;
SN_PS_type qaz2;
SN_PS_type qaz3;
qaz3.ps = (qaz1.ps+qaz2.ps)-(qaz2.ps*qaz1.ps);
```

モードを最適化している時は、コンパイラもまた複合 PS 演算の生成をサポートします。

サポートされる複合演算は以下の通りです。

演算	書式	Gekko オプコード
Multiply Add	x*x+x	ps_madd
Multiply Subtract	x*x-x	ps_msub
Negate Multiply Add	-(x*x+x)	ps_nmadd
Negate Multiply Subtract	-(x*x-x)	ps_nmsub

SN API for PS (SNAPS) 演算

SNAPS は、ベクタータイプをサポートする他の PPC CPU 向けの Motorola AltivecTM 拡張に似たものです。一連のビルトインコンパイラ関数により、どの CPU PS オプコードにも 直接アクセスできます。ネイティブモードのように、すべてのレジスタ配置、ロード、ストアはコンパイラが行います。

コンパイラ中で SNAPS ルーチンは、'__builtin_'(名前)を前につけるという GNU の決まりに従います。さらに、わかりやすさを向上させるために追加のヘッダファイル 'SN_SNAPS.h'で別名の定義が与えられます。

ほとんどの PS オプコードには、条件フラグがオプコードによって更新されるべきことを示すオプションフィールドがあります。これらのオプコードにはこのビットの設定を示すためにビルトイン関数名の後ろに'x'の文字がつけられます(ps_add にたいする ps_addx がその一例です)。

オプコード、結果 1、引数 1	結果 = 関数(ps 引数)	
演算	**ビルトイン**	**SNAPS.h**
ネゲート	__builtin_ps_neg	ps_neg
絶対値	__builtin_ps_abs	ps_abs
負数絶対値	__builtin_ps_nabs	ps_nabs
逆数概数	__builtin_ps_res	ps_res
逆数平方根概数	__builtin_ps_rsqrte	ps_rsqrte

オプコード、結果 1、引数 2	結果 = 関数 (ps 引数 1、引数 2)	
演算	**ビルトイン**	**SNAPS.h**
足し算	__builtin_ps_add	ps_add
引き算	__builtin_ps_sub	ps_sub
割り算	__builtin_ps_div	ps_div
掛け算	__builtin_ps_mul	ps_mul
マージハイ	__builtin_ps_merge00	ps_merge00
マージダイレクト	__builtin_ps_merge01	ps_merge01
マージスワップ	__builtin_ps_merge10	ps_merge10
マージロー	__builtin_ps_merge11	ps_merge11
掛け算スカラーハイ	__builtin_ps_muls0	ps_muls0
掛け算スカラーロー	__builtin_ps_muls1	ps_muls1

オプコード、結果 1、引数 3	結果= 関数 (ps 引数 1, ps 引数 2, ps 引数 3)	
演算	**ビルトイン**	**SNAPS.h**
掛け算足し算	__builtin_ps_madd	ps_madd
掛け算引き算	__builtin_ps_msub	ps_msub
負数掛け算足し算	__builtin_ps_nmadd	ps_nmadd
負数掛け算引き算	__builtin_ps_nmsub	ps_nmsub
ベクトル和ハイ	__builtin_ps_sum0	ps_sum0
ベクトル和ロー	__builtin_ps_sum1	ps_sum1
掛け算足し算スカラーハイ	__builtin_ps_madds0	ps_madds0
掛け算足し算スカラーハイ	__builtin_ps_madds1	ps_madds1
選択	__builtin_ps_sel	ps_sel

コンパイラは通常、PS 値を利用するために必要なすべてのロードとストアを生成しますが、以下に挙げたビルトイン関数はロードとストアオプコードに直接アクセスできます。これは通常、定量的なロードとストアを実行するのに必要です。こういった場合、関連する GQR レジスタが正しく設定されていることを確認するのはプログラマの責任です。

ロード オプコード	**結果=関数(void* Val, int W, int I)**	
演算	**ビルトイン**	**SNAPS.h**
定量的ロード	__builtin_ps_l	ps_l
定量的ロード更新付	__builtin_ps_lu	ps_lu

ロード インデックスオプコード	**結果=関数 (void* Val, int O, int W, int I)**	
演算	**ビルトイン**	**SNAPS.h**
定量的インデックスロード	__builtin_ps_lx	ps_lx
定量的インデックスロード更新付	__builtin_ps_lux	ps_lux

ストア オプコード	**関数(void* Res, ps Val, int W, int I)**	
演算	**ビルトイン**	**SNAPS.h**
定量的ストア	__builtin_ps_st	ps_st
定量的ストア更新付	__builtin_ps_stu	ps_stu

ストア オプコード	**関数(void* Res, ps Val, int W, int I)**	
演算	**ビルトイン**	**SNAPS.h**
定量的ストア	__builtin_ps_st	ps_st
定量的ストア更新付	__builtin_ps_stu	ps_stu

一般的なプログラミングにおける考慮事項

Gekko CPU の PS モードは、通常の Power PC アーキテクチャに追加されたものです。ハードウェア実行時におけるいくつかの要因により、PS モードコードを書きこむ際には細心の注意が必要です。

PS モードオプコードの主な変更点は、インデックスロードと配置操作用のオフセット範囲が 16 ビットから 12 ビットに減少していることです。これによってグローバルあるいは小データ集積をロードおよび配置するためのコンパイラの最適化が実行できなくなります。グローバル変数にアクセスするには、レジスタにフルアドレスをロードする必要があります。これによって実行能力が左右します。従って可能な限り、グローバル PS モード変数の使用は控えたほうが良いと思われます。

もう 1 つの要因は、浮動小数点レジスタの数値タイプが正しく保存、復元されていなければなりません（すなわち、通常の浮動オプコードを使って PS モードを保存したり、その逆を行

なうことはできません)。これを簡単に解決する方法は、プログラムの中でダブル浮動タイプと PS モードタイプを混ぜて利用しないことです。コンパイラは倍精度浮動小数点を使わず、PS モード浮動小数点だけを使用していることを示すのに'-mps-nodf'スイッチを使います。これによりコンパイラは単純に PS モードでレジスタを保存します。

PS モード浮動小数点と倍精度浮動小数点を混在していると、通常の呼ばれた側が保存する関数呼び出しにかなりの負担がかかることになります(すなわち、呼び出された関数は FP レジスタが使用しているものを保存します)。これは callee ルーチンには、FP レジスタが PS あるいは通常の浮動小数点モードのどちらを保存しているかを見分ける機能がないためです。すべての浮動小数点モードにレジスタを 2 回保存させる(該当すると思われるモードに 1 回ずつ)代わりに、PS モードコードをターゲットとするときに、コンパイラが PS モード値を含む浮動小数点レジスタ用の.caller 保存 モードを作成します。calling ルーチンは各浮 動小数点ポイントレジスタの数値タイプを理解しているため、それぞれに 1 回保存するだけで済みます。

例

以下は SNAPS での行列操作ルーチンのコーディング例です。基本アルゴリズムは任天堂 SDK で手書きのアセンブラで利用された構造をそのまま使用しています。GNU アセンブラで考慮しなければならない複雑なアセンブラの制約とパラメータに関する作業が SNAPS では要らなくなることがこのソースからわかります。

テンポラリローカルを利用するのは単に読みやすさを向上するためです。最適化されたビルドではこれらのローカルは、その値がルーチンにいる間中レジスタに保持されるため、コードの生成に影響を与えません。また、すべての単一シングルのロードとストアはコンパイラに任されていることに注意してください。これも裏でコンパイラが使用する ABI を意識する(すなわち何がどのレジスタにあるか)ことなくにユーザが思い通りの単一ペアコードを書ける一因です。「__restrict__」キーワードを利用することでコンパイラがルーチンの出力(マトリクス D)のどれもが入力の別名でないことを前提とすることを保証します(すなわち D は A または B を参照していない)。これが行われないとコンパイラは D の一部が生成されるたび にたくさんの不必要なロードを生成してしまいます。この問題の別な解決方法としては出力をいったん一時的なマトリクスに出力しておいてこれをルーチンの終わりで D にコピーする というアルゴリズムにする方法があります。

```
#include <SN_PS.h>

typedef struct SNMtx
{
  sn_ps m00m01 ;
  sn_ps m02m03 ;
  sn_ps m10m11 ;
  sn_ps m12m13 ;
  sn_ps m20m21 ;
  sn_ps m22m23 ;
} ;

SNMtx A ;
SNMtx B ;
SNMtx D ;

void SNMtx_Concat ( const SNMtx& __restrict__ A,
                    const SNMtx& __restrict__ B ,
                          SNMtx& __restrict__ D)
{
```

```
  register sn_ps Unit01 = PS_BUILD(1.0f,1.0f) ;
  register sn_ps temp1 ;

  temp1 = ps_madds0(B.m20m21, A.m02m03, ps_muls0(B.m00m01,
A.m00m01)) ;
  D.m00m01 = ps_madds1(B.m10m11, A.m00m01, temp1) ;

  temp1 = ps_madds1(B.m12m13, A.m00m01, ps_muls0(B.m02m03,
A.m00m01)) ;
  temp1 = ps_madds1(Unit01, A.m02m03, temp1);
  D.m02m03 = ps_madds0(B.m22m23, A.m02m03, temp1) ;

  temp1 = ps_madds1(B.m10m11, A.m10m11, ps_muls0(B.m00m01,
A.m10m11)) ;
  D.m10m11 = ps_madds0(B.m20m21, A.m12m13, temp1) ;

  temp1 = ps_madds1(B.m12m13, A.m10m11, ps_muls0(B.m02m03,
A.m10m11)) ;
  temp1 = ps_madds0(B.m22m23, A.m12m13, temp1) ;
  D.m12m13 = ps_madds1(Unit01, A.m12m13, temp1) ;

  temp1 = ps_muls0(B.m00m01, A.m20m21) ;
  temp1 = ps_madds1(B.m10m11, A.m20m21, temp1) ;
  D.m20m21 = ps_madds0(B.m20m21, A.m22m23, temp1) ;

  temp1 = ps_muls0(B.m02m03, A.m20m21) ;
  temp1 = ps_madds1(B.m12m13, A.m20m21, temp1) ;
  temp1 = ps_madds0(B.m22m23, A.m22m23, temp1) ;
  D.m22m23 = ps_madds1(Unit01, temp1, temp1) ;
} ;
```

このルーチンが最適化指定でコンパイルされた場合には次のコードが生成されます。

```
       .globl SNMtx_Concat__FRC5SNMtxT0R5SNMtx
       .type  SNMtx_Concat__FRC5SNMtxT0R5SNMtx,@function
SNMtx_Concat__FRC5SNMtxT0R5SNMtx:
        psq_l %f10,0(%r3),0,0         # 28    *movps_internal1/2
[length = 4]
        lis %r9,.LC0@ha               # 16    elf_high
[length = 4]
        psq_l %f13,0(%r4),0,0         # 26    *movps_internal1/2
[length = 4]
        la %r9,.LC0@l(%r9)            # 17    elf_low
[length = 4]
        psq_l %f11,8(%r3),0,0         # 37    *movps_internal1/2
[length = 4]
        psq_l %f12,32(%r4),0,0        # 34    *movps_internal1/2
[length = 4]
        ps_muls0  %f13,%f13,%f10      # 29    b_muls0ps3
[length = 4]
        psq_l %f0,16(%r4),0,0         # 49    *movps_internal1/2
[length = 4]
        ps_madds0  %f12,%f12,%f11,%f13# 40    b_madds0ps3
[length = 4]
        psq_l %f9,0(%r9),0,0          # 19    *movps_internal1/2
[length = 4]
        ps_madds1  %f0,%f0,%f10,%f12  # 54    b_madds1ps3
[length = 4]
        psq_st %f0,0(%r5),0,0         # 56    *movps_internal1/3
[length = 4]
        psq_l %f10,0(%r3),0,0         # 65    *movps_internal1/2
[length = 4]
```

```
        psq_l %f12,8(%r4),0,0         # 63    *movps_internal1/2
[length = 4]
        psq_l %f0,24(%r4),0,0         # 71    *movps_internal1/2
[length = 4]
        ps_muls0  %f12,%f12,%f10      # 66    b_muls0ps3
[length = 4]
        psq_l %f11,8(%r3),0,0         # 87    *movps_internal1/2
[length = 4]
        psq_l %f13,40(%r4),0,0        # 99    *movps_internal1/2
[length = 4]
        ps_madds1  %f0,%f0,%f10,%f12  # 76    b_madds1ps3
[length = 4]
        ps_madds1  %f0,%f9,%f11,%f0   # 90    b_madds1ps3
[length = 4]
        ps_madds0  %f13,%f13,%f11,%f0 # 105   b_madds0ps3
length = 4]
        psq_st %f13,8(%r5),0,0        # 108   *movps_internal1/3
[length = 4]
        psq_l %f10,16(%r3),0,0        # 117   *movps_internal1/2
[length = 4]
        psq_l %f13,0(%r4),0,0         # 114   *movps_internal1/2
[length = 4]
        psq_l %f12,16(%r4),0,0        # 123   *movps_internal1/2
[length = 4]
        ps_muls0  %f13,%f13,%f10      # 118   b_muls0ps3
[length = 4]
        psq_l %f11,24(%r3),0,0        # 141   *movps_internal1/2
[length = 4]
        psq_l %f0,32(%r4),0,0         # 138   *movps_internal1/2
[length = 4]
        ps_madds1  %f12,%f12,%f10,%f13# 129   b_madds1ps3
[length = 4]
        ps_madds0  %f0,%f0,%f11,%f12  # 144   b_madds0ps3
[length = 4]
        psq_st %f0,16(%r5),0,0        # 147   *movps_internal1/3
[length = 4]
        psq_l %f10,16(%r3),0,0        # 157   *movps_internal1/2
[length = 4]
        psq_l %f12,8(%r4),0,0         # 154   *movps_internal1/2
[length = 4]
        psq_l %f11,24(%r4),0,0        # 163   *movps_internal1/2
[length = 4]
        ps_muls0  %f12,%f12,%f10      # 158   b_muls0ps3
[length = 4]
        psq_l %f13,24(%r3),0,0        # 181   *movps_internal1/2
[length = 4]
        psq_l %f0,40(%r4),0,0         # 178   *movps_internal1/2
[length = 4]
        ps_madds1  %f11,%f11,%f10,%f12# 169   b_madds1ps3
[length = 4]
        ps_madds0  %f0,%f0,%f13,%f11  # 184   b_madds0ps3
[length = 4]
        ps_madds1  %f13,%f9,%f13,%f0  # 198   b_madds1ps3
[length = 4]
        psq_st %f13,24(%r5),0,0       # 201  *movps_internal1/3
[length = 4]
        psq_l %f10,32(%r3),0,0        # 210   *movps_internal1/2
[length = 4]
        psq_l %f13,0(%r4),0,0         # 207   *movps_internal1/2
[length = 4]
        psq_l %f12,16(%r4),0,0        # 220   *movps_internal1/2
[length = 4]
        ps_muls0  %f13,%f13,%f10      # 211   b_muls0ps3
```

```
[length = 4]
        psq_l %f11,40(%r3),0,0        # 238   *movps_internal1/2
[length = 4]
        psq_l %f0,32(%r4),0,0         # 235   *movps_internal1/2
[length = 4]
        ps_madds1  %f12,%f12,%f10,%f13# 226   b_madds1ps3
[length = 4]
        ps_madds0  %f0,%f0,%f11,%f12  # 241   b_madds0ps3
length = 4]
        psq_st %f0,32(%r5),0,0        # 244   *movps_internal1/3
[length = 4]
        psq_l %f10,32(%r3),0,0        # 254   *movps_internal1/2
[length = 4]
        psq_l %f13,8(%r4),0,0         # 251   *movps_internal1/2
[length = 4]
        psq_l %f12,24(%r4),0,0        # 264   *movps_internal1/2
[length = 4]
        ps_muls0  %f13,%f13,%f10      # 255   b_muls0ps3
[length = 4]
        psq_l %f0,40(%r4),0,0         # 279   *movps_internal1/2
[length = 4]
        psq_l %f11,40(%r3),0,0        # 282   *movps_internal1/2
[length = 4]
        ps_madds1  %f12,%f12,%f10,%f13# 270   b_madds1ps3
[length = 4]
        ps_madds0  %f0,%f0,%f11,%f12  # 285   b_madds0ps3
[length = 4]
        ps_madds1  %f9,%f9,%f0,%f0    # 298   b_madds1ps3
[length = 4]
        psq_st %f9,40(%r5),0,0        # 301   *movps_internal1/3
[length = 4]
        blr                           # 316   return
[length = 4]
```

ゲームキューブプログラミングのヒント

ここで、ゲームキューブのゲームをビルドする際に役立ついくつかのヒントをあげておきましょう。

1. ウィルスチェックソフトの中には、作成したすべてのファイルをスキャンするものや、実行ファイルや DLL を、実行するたびにスキャンするものがあります。こうした作業はビルド時間を大幅に遅らせ、パフォーマンスが 3 分の 1 以上低下してしまう場合があります。もしシステムにウィルスがいないと確信でき、会社の規則でウィルスチェッカの設定を変えられない状況でなければ、ビルド時間を短くするため、このタイプのファイルをウィルスチェックの対象から外すことをお勧めします。このようなウィルスチェック方法の変更が禁止されていないかどうか、システム管理者に確認して下さい。

2. ドルフィン SDK をインストールすると、SDK のインストール場所を自動的に検出するために makefile で使用するいくつかの環境変数が作成されます。以下の例では、ベースドライブやインストールパスにアクセスするための makefile 変数を宣言しています。

```
empty=
space=$(empty) $(empty)
DOLPHIN_DRIVE = $(word 1,$(subst /,$(space),$(DOLPHIN_ROOT)))
```

```
DOLPHIN_SDK_ROOT=$(subst $(space),\$(space),$(subst \,/,$(subst
//$(DOLPHIN_DRIVE)/,$(DOLPHIN_DRIVE):/,$(DOLPHIN_ROOT))))
```

3. 以下の makefile の引用部分では、使用している開発ハードウェアに適したデバッグモードライブラリを選択する方法を示しています。

```
AMCDIR = $(subst $(space),\$(space),$(subst \,/,$(AMCDDKBIN)))
ODEDIR = $(subst $(space),\$(space),$(subst \,/,$(ODEMUSDKBIN)))
ifdef AMCDDKBIN
DEBUGLIBS = $(AMCDIR)/../lib/AmcExi2.a
$(INSTALL_ROOT)/lib/amcnotstub.a
else
DEBUGLIBS = $(INSTALL_ROOT)/lib/amcstubs.a
endif # AMCDDKBIN

ifdef ODEMUSDKBIN
DEBUGLIBS += $(ODEDIR)/../lib/odemuexi2.a
$(INSTALL_ROOT)/lib/odenotstub.a
else  # ODEMUSDKBIN
DEBUGLIBS += $(INSTALL_ROOT)/lib/odemustubs.a)
endif # ODEMUSDKBIN
```

以下の makefile 引用部分では、DOS 形式のパスを makefile 形式のパスに変換する方法を示しています。

```
MAKE_PATH = $(subst $(space),\$(space),$(subst
\,/,$(DOS_PATH)))
```

4. asm 指令に馴染んでいる場合は、レジスタ用に「r」ではなく「b」(「base」レジスタ)を使用します。この PPC システム上で「r」を使用すると、r0 が使用 されることがあり、それを定数 0 とみなすオペレータもあれば、0 番目のレジスタとみなすオペレータもあり、常に 0 の値に設定されません。

トラブルシューティング

警告を発しない C コンパイラ

C コンパイラの機能には、デフォルトで設定されていない、いくつかの特別の 警告設定があります。さらに、「すべてに警告 (-Wall)」オプション設定によっては、設定されません。ビルドコマンドの '-W' スイッチで提供されている、'特別' 警告によって有効となります。 入力例は以下の通りです。

```
ngccc -W -Wall myfile.c
```

「未知のエスケープシーケンス」エラー

Shift-JIS 符号化文字を含むソースコードをコンパイルするときに、次の警告が表示されることがあります;

```
"unknown escape sequence: '\' followed by char code X"
```

これは、Shift-JIS 複数バイト文字に、C プリプロセッサがコントロールコードとして解釈 する、0x5C ('\') が含まれる場合があるためです。

この問題を解決するには、2 つの方法があります:

1. S-JIS 0x5C ('\') 文字のとなりに、余分な 0x5C ('\') 文字を挿入します。これによって、C プリプロセッサが、コントロールコードとしてではなく、文字コードとして解釈するようになります。

2. 次の Windows、または makefile 環境変数を作成します:

   LANG=C-SJIS

[このページは意図的に空白にされています]

第 6 章: ProDG アセンブラ

SN Systems のアセンブラを使用する

ニンテンドーゲームキューブ用 ProDG ビルドツール群には、SN Systems のアセンブラ、ngcas を含有します。

ngcas のコマンドラインオプションと命令は、GNU アセンブラと互換性がありますが、GNU コマンドラインスイッチの中には、認識されなかったり、エラーを起こすものもあります。詳細については89ページの「サポートされていないアセンブラオプション」と89ページの「サポートされていないアセンブラダイレクティブ」を参照してください。

さらに、-sn オプションを指定すると、SN Systems 命令にアクセスできるようになり、範囲指定やレジスタの名前付けなど、優れた機能を使用できます。51ページ「SN Systems 命令」をご覧下さい。

コマンドライン書式

SN Systems のゲームキューブアセンブラである ngcas は、GNU アセンブラそのものとして直接その機能を実行し、かつ GNU アセンブラと完全に互換性があります。ngcas を使用するには、以下のコマンドラインで直接呼び出す必要があります。

```
ngcas <options> <input file>
```

このコマンドラインでは、プログラム名の後に、オプションやファイル名が含まれることがあります。

各オプションは、アセンブラの動作を変更します。他のオプションの変更に影響するオプションはありません。オプションは、「-」の後に 1 つ以上の文字が続きます。ここでは文字の 大文字・小文字の違いが重要となることに留意して下さい。オプションはどの順序でも現れ、前、後、またはファイル名間に現れることもあります。

「-- (ハイフン 2 回)」は、それ自体が、アセンブル対象ファイルの 1 つとして、標準入力ファ イルを明示しています。

オプションの中には、後に続くファイル名が正確に 1 つと決まっているものがあります。こうしたファイル名は、オプションの文字の後に直接続けるか、次のコマンド引数とします。これら 2 つのコマンドラインは、以下に示す通り、同じものとなります。

```
ngcas -o my-object-file.o mumble.s

ngcas -omy-object-file.o mumble.s
```

以下の表は、使用可能なオプションを一覧表示したものです。

オプション	アクション
--defsym sym=value	整数のシンボルを定義する
-G<size>	.sdata/.sbss に収録するデータ項目のサイズを設定する
--gdwarf1	DWARF1 デバッグ情報を作成する
--help	ヘルプを表示する
-I <directory>	収録ファイルを検索するディレクトリを指定する
-keep-constants	シンボル表の常数を保持。デフォルトではシンボル表から常数を除外
--keep-locals	– L と同じ
-L	ローカルのシンボル情報を出力する
-MD<filename>	GNU make 依存ファイルを作成する
-o <output file>	出力ファイル名を指定する
-sn	SN Systems の拡張機能を使用可能にする。詳しくは、51 ページ「SN Systems 命令」参照
--stdout	エラーメッセージを stderr にではなく、stdout に印刷する
--version	バージョン情報を表示する
-W	警告を無効にする

C コンパイラを通じて GNU アセンブラが呼び出された時、アセンブラ オプションと同 じメカニズムを使用して、-Wa オプションでアセンブラに引数を引き渡せます。アセンブラの 引数は、それぞれ (および-Wa と) コンマで区切る必要があります。多くのコンパイラ コマンドラインオプションは、コンパイラによって自動的にアセンブラに引き渡されるため、通常この-Wa メカニズムを使用する必要はありません。(-v オプションを使用して GNU コンパ イラを呼び出し、アセンブラを含む各コンパイルパスに引き渡したオプションを正確に知ることができます。)

アセンブラ命令

以下のリストでは、ngcas が認識するすべてのデフォルト命令を、簡潔に解説しています。このリストには、GNU のマニュアルで解説している標準の GNU 命令は含まれていません。

命令	解説
.data	.data セクションに切り換える
.quad	128 ビットのデータ文を指定する
.word	32 ビットのデータ文を指定する

SN Systems 命令

以下の表は、コマンドラインで–sn スイッチを指定すると有効になる SN Systems 命令の一覧です。

注: 変数__rs で、直接 rs カウンタの値を得ることができます。

命令	解説
.endscope	.scope (下記)参照
.equr newreg, reg	レジスタに別の名称を指定する。サンプルコードは、51ページの「レジスタ名称指定およびレジスタフィールド」を参照
.incbin "filename"	「ファイル名」を実際にはアセンブルせずにインクルード
.rpalloc poolname, newreg, newreg, ...	指定されたレジスタプールである poolname からレジスタ値を取得して、各 newregs のレジスター致作業を行なう
.rpfree poolname, reg, reg, ...	指定された各 reg を poolname プールに戻し、そのレジスタ名に未定義マークをつけて、誤ってもう一度使わないようにする
.rpinit poolname, reg, reg, ...	指定した poolname のレジスタプールを作成し、そのプールから指定できるレジスタのリストを作成する
.rsb name,count	rs カウンタを 1 バイトを境界とするよう調節し、rs カウンタ値を名称に割当て、rs カウンタをカウントごとに進める
.rsd name,count	rs カウンタをダブルワード境界として整列、rs カウンタ値を name に割り当て、およびカウント* size で rs カウンタを提出する
.rsh name,count	rs カウンタをハーフワード境界として整列、rs カウンタ値を name に割り当て、およびカウント* size で rs カウンタを提出する
.rsq name,count	rs カウンタをクアッドワード境界として整列、rs カウンタ値を name に割り当て、およびカウント* size で rs カウンタを提出する
.rsreset {expression}	rs カウンタを 0、または数式が指定された場合はその数式の値に設定する
.rsw name,count	rs カウンタをワード境界として整列、rs カウンタ値を name に割り当て、およびカウント* size で rs カウンタを提出する
.scope .endscope	ローカルラベルの範囲を定める。「@」という文字で始まるラベルは、全てこの範囲内に置かれる。サンプルコードは、52ページの「範囲の指定」を参照。

レジスタ名称指定およびレジスタフィールド

SN Systems 製.equr 命令をレジスタおよびレジスタフィールド名称設定に使用します。
例:

```
.equr MyRegister, %r1
.equr MyFloat   , %f3
```

レジスタラベルも同一のソースファイルで再定義可能。例えば、

```
.equr AREGISTER, %r0
...
.equr AREGISTER, %r1
```

範囲の指定

以下のサンプルコードセグメントは、範囲指定命令である.scope / .endscope の 使用法を示します。

```
.scope
@L1:
   bnez  $4,@L1
   add   $4,-1
.endscope

.scope
@L1:
   bnez  $5,@L1
   add   $5,-1
.endscope

Scopes can be nested.

e.g.

.scope
@L1:
   nop
.scope
@L1:
   bnez  $5,@L1  // goes to second $L1
   add   $5,-1
.endscope
   bnez  $4,@L1 // goes to first $L1
   add   $4,-1
.endscope
```

ある範囲から、ネスティングされた範囲のローカルラベルはアクセスできなくなります。.scope/.endscope の 2 つに囲まれていない全ての ローカルラベルは、 グローバル範囲にあります。つまり、ローカルでないラベル名の間には、範囲が設定されません。

アドレスモード

label@ha	サインが拡張された下位アドレスに合わせて修正された上位アドレス
label@h	サイン拡張修正のない上位アドレス
label@l	下位アドレス
label@sda21(0)	小さなデータレジスタと関連するアドレス。小さなデータセクションでラベルを宣言する必要がある。使用するレジスタの選択はリンカが行なう。

入力ファイル

*ソースプログラム*または*ソース*は、アセンブラの実行時に入力されるプログラムを記述しています。このプログラムは、 1 つもしくはそれ以上のファイルにある場合があり、ソース がいかにパーティションされていてもソースの意味を変更するものではありません。

ソースプログラムは、指定された順序通りに並べられた、全てのファイルのテキストを凝縮したものです。

アセンブラを実行する度に、正確に 1 つのソースプログラムがアセンブルされます。ソースプログラムが 1 つ以上のファイルに作成されます（標準入力もファイルの一つです）。

アセンブラに 1 つ以上の入力ファイル名を含むコマンドラインを与えるとします。入力されたファイルは左から右へ読み取られます。特定の意味を持たないコマンドライン引数は、(どこにあっても) 入力ファイル名として認識されます。

アセンブラにファイル名を与えないと、アセンブラは標準入力から入力を読み取ろうとします。これは通常、端末のキーボード入力です。そこで、`<Ctrl+D>`をタイプして、アセンブルするソースプログラムがないことをアセンブラに知らせる必要があります。

- コマンドラインで、標準の入力ファイルを明示的に示す必要がある場合は、「--」スイッチを使用します。

ソースが空の場合は、アセンブラは小さな空のオブジェクトファイルを作成します。

ファイル名と行番号

入力ファイル (複数でも可) 内の行の場所を特定する方法は 2 つあり、どちらもエラーメッセージをレポートする際に使用できます。1 つは、物理ファイル中の行番号で、もう 1 つは、「論理」ファイル中の行番号です。54ページの「エラーおよび警告メッセージ」をご覧ください。

*論理ファイル*とは、アセンブラに与えられるコマンドラインで指定されるファイルです。

*論理ファイル*は、アセンブラ命令が明示的に宣言する名称に過ぎず、物理ファイルとは何の関連もありません。論理ファイル名は、アセンブラソース自体が他のファイルから合成されている場合に、エラーメッセージが元のソースファイルを反映する際に役立ちます。命令.app-file string が使用されます。

出力(オブジェクト)ファイル

アセンブラを実行する度に、1 つのオブジェクトファイルが作成されます。慣例的に、オブジェクトファイル名の最後には拡張子 .o が付くため、<file>.s をアセンブルすると、デフォルトのオブジェクトファイル名は <file>.o となります。ただし、-o オプションを使用すると、別の名称を付けられます。

オブジェクトファイルは、リンカの入力ファイルとなります。オブジェクトファイルには、アセンブルされたプログラムコード、リンカがアセンブルされたプログラムを実行ファイルに統合する際に使用する情報、デバッガのシンボル情報(省略可能)が含まれています。

エラーおよび警告メッセージ

アセンブラが、標準のエラーファイル (通常使用している端末) に、警告やエラーメッセージを書き込む場合があります。コンパイラがアセンブラを自動的に実行している場合は必要ありません。警告は、アセンブル作業が続けられるようプログラムに欠陥があるとみなし、一方、エラーはアセンブル作業の続行を妨げる致命的な問題を報告します。

警告メッセージは、以下のようなフォーマットで表されます

```
<file_name>:(<line_number>) : Warning: <message>, "<line of
code>"
```

- 論理ファイル名が(.app-file string 命令を使用して)与えられている場合は、その<file_name>がファイル名として使用され、論理ファイル名が与えられていない場合は、現在の入力ファイル名が使用されます。
- 論理行番号が(.line line-number 命令で)与えられている場合は、それを使用して表示される行番号<line_number>が算出され、論理行番号が与えられていない場合は、現在のソースファイルの実際の行が表示されます。
- テキスト"<line of code>"が問題のあるソースの行を表示します。

エラーメッセージは、以下のようなフォーマットで表示されます。

```
<file_name>:(<line_number>) : Error: <message>, "<line of
code>"
```

- ファイル名と行番号は、警告メッセージの場合と同様の方法で表示されます。

シンボル名

GNU アセンブラでのシンボル名の形成に関する詳細な情報は、GNU の Web サイト http://www.gnu.org/manual/gas-2.9.1/html_mono/as.html - SEC45 でオンラインドキュメントを参照してください。

ローカルシンボル

アセンブラの書き込みには、いくつかのタイプのローカルシンボルを使用できます。どれを使う必要があるかを、判別するのが難しい場合があります。

ファイル有効範囲でローカルなシンボルがあります。これらは、C コードで "static" とし て宣言された、変数または関数と同等です。他のオブジェクトファイルは、これらのシンボルに参照できません。他のファイルには、同じ名前の独自のローカルシンボルがある場合があります。これによって、リンク時に問題が生じることはありません。

すべてのラベルは、デフォルトでローカルと見なされ、".global" のような、コンパイラ 命令によって、グローバルシンボルに変換されます。

アセンブラとリンカは、ファイルと関連するコンテキストを考慮する際に、通常これらのシンボルを有効範囲に保存します。たとえば、オブジェクト内で、コードをデバッギングするときには、ローカルシンボルは有効範囲内にあり、デバッガがこれらを評価します。

コードフロー コントロールで使いたいと希望するシンボルには、命名規則がありますが、デバッギングとの関連性はありません。これらのシンボルには".L" のプレフィックスがつきます。 もう一 つの方法として、コマンドオプションの -sn を使って、SN 拡張子を有効にすると、"@" プレフィックスを使うこともできます。これらのシンボルは、アセンブリまたはリンキングの目的で使われますが、不要となった場合には破棄されます。また、最終的なエグゼクタブルには表示されません。シンボルは、アセンブリファイル内で、固有の名前を維持することが必ず必要です。コンパイラは、ループの始めにアドレスと関連した論理的な名前のないループには、ローカル プレフィックスの付いたラベルを使います。希望する場合は、コマンドライン でシンボルを保存するために、 "-L" または "--keep-locals" を使用してください。アセ ンブラを手書きで書き込む場合は、デバッギングのフローコントロールに関係するラベルを保存するようお勧めします。この場合、プレフィックスの付かない普通ラベルを使ってください。ただし、コンパイラと同様に、最終シンボル表から、これらのラベルを破棄するには、ローカルラベル プレフィックスを使う場合もあります。

レジスタリファレンス

ニンテンドーゲームキューブに搭載の Gekko CPU 用 SN アセンブラはレジスタ番 号およびレジスタ名の両方を認識します。

コンパイラはデフォルトでシンプルなレジスタ番号を設定しますが、アセンブラソースファイル結果内で読み込みづらいという点で不利です。さらに、二つのレジスタ名スタイルの混合時に問題が生じることにもなりかねません。特にレジスタ番号「0」と即値「0」でよく起こります。

このような混乱を避けるため、アセンブラはレジスタ名慣例がオプコードと混合されたときに警告を出します。読み込みを少しでも簡単にするため、インラインアセンブラ入りのコードをコンパイルするときには、コンパイラにパラメータ'-mregnames'を使用して完全レジス タ名を使用するように指示することをお勧めします。

レジスタ数

レジスタ番号はオプコードにレジスタを表すシンプルな数字のことです。しかし、レジスタ形式は区別しません。例えば通常使用のレジスタと浮動小数点レジスタと区別するため、オプコードのモードを見分ける必要があります。

```
add             1,2,3
fadd            1,2,3
```

レジスタ名

レジスタ名は、書式「%」に続くレジスタの名前、という体裁になっています。通常使用のレジスタは r0 から r31 という名前を使用し、浮動小数点レジスタは f0 から f31 を使用します。例えば、

```
add             %r1,%r2,%r3
fadd            %f1,%f2,%f3
```

レジスタ名とインラインアセンブラパラメータ

GNU コンパイラは C/C++パラメータをアセンブラブロックに引き渡す機能を持っています。慣例に従い、<n>がパラメータ番号の場合は'%<n>'とします。しかし、これは明らかにレジスタ名に使われた「%」記号とクラッシュします。このような理由で、アセンブラブロックにレジスタ名を書き込む場合は、「%」記号を二回拡張し、パラメータを区別する必要があります。例えば、'%r0'は'%%r0'となります。また、

```
asm ( " add %%r1,%%r2,%%r3 ") ;
```

レジスタとインプット/出力/クラバーリスト

GNU アセンブラブロックはインプット、出力、クラバーリスト、およびそのモードを示すパラメータを含有することがあります。状況に応じて以下のようにモードパラメータを書き換える必要があります。

レジスタ対ベースモード

Gekko CPU は、もっとも効率のよいアドレス作成のためにオフセットも付加し、通常使用レジスタをベースアドレスとして使用します。しかし、通常使用レジスタ r0 はこのベースオフセットモードでは使用できません。ベースオフセットが必要なパラメータモードに引き渡す際はレジスタクラスを表す'r'ではなく、'b'をベースレジスタクラスとして使用して下さい。

メモリモードと単一ペアオプコード

'm'パラメータクラスはベースレジスタ、および 16 bit オフセットとして作成されるメモリ配置を定義します。しかし、単一ペアロードおよびストアオプコードは完全 16 ビットオフセットを使用できません。'm'クラスがコンパイラに指令するのではなく、手動で有効なアドレスを作成する必要があります。

例えば下記の単一フロートロードは 16-bit オフセットを使用できます。

```
asm volatile ("lfs %%f1, %0 " : : "m" (fvar) : "fr1" ) ;
```

以下の警告が出力された場合:

```
Warning : Register number without %r prefix used
```

これを無視するか、あるいは次のようにステートメントを変更してください:

```
asm volatile ("lfs 1, %0 " : : "m" (fvar) : "fr1" ) ;
```

ステートメントは、次のアッセンブラを生成します:

```
lis %r9,fvar@ha
lfs %f1,fvar@l(%r9)
```

しかし、単一ペアでは、以下のようなエラーが返されます。

```
Error : This patch type is not allowed for 12bit offset;
```

代わりに、単一ペアの完全アドレスを一時レジスタとしてロードする必要があります。

```
asm volatile ("psq_l %%f1, %0, 1, 0 " : : "m" (&fvar) :
"fr1" ) ;
```

が以下を作成します。

```
lis   %r9,.LC0@ha
la    %r9,.LC0@l(%r9)
psq_l %f1, 0(%r9), 1, 0
```

クラバーリストのレジスタ名

さらに複雑な事例として、インプット，出力、クラバーリスト上のレジスタ名に、コンパイラが若干異なる書式を使用する、ということを提示します。これらはアセンブラではなく、コンパイラのコマンドであるため、独自の内部レジスタ名慣例を使用します。これらリストでレジスタ名を定義する際は'%'は必要ありません。

インラインアセンブラのデバッギング

インラインアセンブラブロックが正しいかどうかをチェックする最も簡単な方法は、コンパイラからアセンブラの'.s'出力ファイルファイルを調べることです。中間ファイルを削除しないようビルドツールを調べるため、'-save-temps'パラメータを使用します。

作成された'.s'ファイルに読み込み可能なコメントを、'#'コメント文字を使って埋め込みます。また、C/C++パラメータ'%<n>'を、お使いの変数に正しく選択されたインプット、および出力モードでさらにチェックするときもこのコメントを使用できます。

[このページは意図的に空白にされています]

第 7 章: ProDG リンカ

ngcld リンカを使用する

この章では、ProDG for Nintendo GameCube リンカの機能と有効性について 説明いたします。以下が、リンカの主要な機能です:

- ProDG デバッガ ProDG Target Manager、および dsedb と互換性のある ELF イメージを生成
- ProDG デバッガおよび dsedb と互換性のあるデバッグ情報を生成
- GNU ツールでビルドされた Nintendo GameCube オブジェクトファイルおよびライブラリをサポート
- テンプレート、グローバルなオブジェクトの構築・破棄、例外などを含む C++をサポート
- メッセージおよび MAP ファイルに含まれるシンボル名を再構築
- イメージから未使用および重複関数を削除
- ProDG ツールライセンスが必要(ProDG デバッガと同じライセンス)
- GNU linker ld よりも更に速い

必要メモリ

リンカスワッピングを防ぐため、ご使用のシステムには最低 256MB の RAM を確保してください。また、無理のないリンクのため経験則として、ngcld メモリはリンク中のオブジェクト、およびライブラリファイルのトータルサイズの最低半分に該当する量が必要と言えます。

リンカ スクリプト

ProDG は'ld format'リンカスクリプトをデフォルトでインストールします。これはす べての サンプルに有効です。このスクリプトは bin/ngc.ld としてインストールされます。ProDG リンカである ngcld は、大半の'ld format'リンカスクリプト・ディレクティブをサポートします。

リンカスクリプトのフォーマットに関する詳細情報は、GNU の Web サイト **www.gnu.org**. で、linker ld ドキュメントを参照してください。

GNU linker ld でサポートされているが、ngcld にはサポートされていないスクリプト ファイル命令の完全一覧は、92 ページの「ngcld でサポートしていないスクリプトファイル命令」を参照してください。

セクション

コンパイラの出力で表示される可能性のあるセクションの完全一覧は以下の通りです。

セクション	用途
.text	プログラムコード
.data	初期化された変数
.rodata	文字列などの読み出し専用データ
.bss	初期化されていない変数
.sdata	初期化された変数 (小データ)
.sbss	初期化されていない変数 (小データ)

リンカスクリプトでファイルを参照する

リンカスクリプト内のオブジェクトファイル名にワイルドカードがない場合、リンクコマンドラインに現れなくてもリンクに必要であるとみなされます。従って、もしリンカがオブジェクトを見つけられない場合はリンクが落ちます。例えば、

```
mysection :
{
      foo.o(.text)
}
```

この場合、foo.o にワイルドカード('*'または'?')がないのでリンクに付加されます。もしリンカが foo.o を見つけられない場合はリンクが落ちます。

コマンドラインに明示的にリストされている foo.o をリンクだけに足したい場合は、スクリプトに若干手を加え、ワイルドカードを含むようにします。例えば、

```
mysection :
{
      foo.o*(.text)
}
```

ngcld コマンドライン書式

T 以下が、ngcld リンカ コマンドライン書式です:

```
ngcld [switches] [files]
```

switches	以下のセクションで解説する全てのコマンドライン スイッチ。これらの スイッチの前には、ハイフン(「-」)を付ける必要がある。
files	オブジェクトファイルの場合と、ライブラリの場合がある。リンカは、リンカ

スクリプトで指定されたファイルに加えて、これらのファイルを使用する。

ngcld スイッチ

コマンドラインスイッチは、前にハイフン(–)を付け、以下に示すもののいずれかを使用するものとします。

スイッチ	アクション
-comment	コメントセクションを保存
-d, -dc or -dp	リンカに、部分リンクであっても共通データを割り当てるよう指令
-debug <file>	デバッグログを<file>に出力する。
-defsym <symbol>=<value>	<symbol> と <value> を定義
-e <entry> or --entry=<entry>	<entry> に入力ポイントを設定
--fix-sda	小データ領域参照の問題の修復を試みる。
-G<size> or --gpsize=<size>	小さなデータオブジェクトの閾値を<size>バイトに設定する。すなわち、<size>バイト分のデータはオブジェクトとし、それ以下のデータを小さなデータセクションとする
-I, -r, -relocatable or -Ur	部分リンク実行をリンカに指令。
-l<library>	ライブラリファイル「lib<library>.a」をリンクに含める。たとえば–lc と入力すると、リンカは libc.a を検索する
-L <path> or -library-path <path>	ライブラリを検索するディレクトリのリストにこのパスを追加する
-Map <mapfile>	<mapfile>内にマップファイルを生成する
-no-keep-memory	リンク時に少ないメモリを使用する
-o <file> -output <file>	出力先ファイルとして<file>を使用。
-print-map or -M	stdout にマップファイルを印刷する
-report-unused	未使用関数リストをファイル statcov.txt に書き込む
-S or -strip-debug	出力からのデバッグ情報をストリップする
-S-lib	ライブラリからのデバッグ情報をストリップする
-s or -strip-all	シンボルをストリップし、出力からの情報をデバッグする
-s-lib	シンボルをストリップし、ライブラリからの情報をデバッグする

-sn-force-section <section>	このセクションが空欄でも出力する
-sn-full-map	マップファイル中に追加の情報(たとえばスタティック変数)を入れる
-sn-no-dtors	リンカがデストラクタを未使用として記す。-strip-unused または -strip-unused-data に関連して使用された場合は、未使用のデストラクタコードがゲームから削除される
-sn-overlays	デバッガにオーバレイコードのデバッグをさせる
-start-group <libs> -end-group	<libs>内の任意のライブラリで定義されているシンボルを、そのシンボルが定義・使用されている順番に関わらず、解除できるようにする。64 ページの「「未定義シンボルへの参照」エラー」を参照
-strip-unused	未使用関数を破棄する
-strip-unused-data	未使用関数コードとデータを破棄する
-T <file>	<file>をリンカ スクリプトとして使用する
-t or -trace	ファイルが開かれると同時に名前を印刷する
-Tbss <addr>	<addr>を bss セクションのアドレスとして設定する
-Tdata <addr>	<addr>を data セクションのアドレスとして設定する
-Ttext <addr>	<addr>を text セクションのアドレスとして設定する
-x or -discard-all	ローカルシンボルを破棄する

ProDG linker ngcld は、GNU linker, ld, でサポートされている多くのスイッチが、GameCube プラットフォームには適切でないため、これらのすべての スイッチをサポート しません。ngcld にサポートされていないスイッチの完全一覧は、91 ページの、「サポートされていない ld スイッチ」を参照してください。

小データ領域のサポート

ngcld リンカは、EABI インタフェースが指定する小データ領域の使用法に、厳密には従っていません。ngcld リンカは、R2、R13 のいずれかを使用して、任意の小データ領域にあるオブジェクトを参照します。1 つのレジスタで読み出し専用データを、別のレジスタで読み書き可能データを参照することはありません。

ngcld リンカでは、32K 上方の小データ、_SDA_BASE_、_SDA2_BASE_だけでなく、最大 32K 下方の小データを参照できます。

リンカが利用するこれらの方法は、EABI 完全準拠のリンカでは可能なコード生成を、何ら阻害するものではありません。

リンカスクリプトを修正して、参照可能な小データの量に関する従来の制限を拡大できます。通常、この制限は読み書き可能データが 32K、読み出し専用データが 32K です。この機能を使えば、これら 2 種類のデータ合計で、128K というサイズの小データが参照可能に

なります。総容量 128K という小データを実現するには、すべての小データセクションがリンカスクリプトで共存する必要があります。_SDA_BASE_は、最下位にある小データアドレスの 32K 上方に設定されます。一方、_SDA2_BASE_は、最下位にある小データアドレスの 192K 上方に設定されます。

注: 小データ領域最適化は再配置可能モジュールとは使用できません

小データを使用する目的

レジスタを直接参照するよりも、レジスタに関連付けされたデータを参照するほうが、処理を高速化できます。しかし、この方法で参照できるデータ量には、制限があります。

小データセクションの適用範囲を調節する

コンパイラは、データを配置するセクションを、データのサイズに基づいて選択します。

データ項目を強制的に data または bss セクションに配置するには、以下のように、その項目をセクション属性と共に宣言します。

```
int rarely_used attribute(section(".bss"));
```

変数を小データセクションに強制的に配置するには、以下のように、その変数を sbss または sdata セクションにあるものとして宣言します。

```
char fast_array[10] attribute(section(".sbss"));
```

初期化されていないデータは bss セクションに、初期化されているデータは data セクションに配置する必要があります。

- データをセクションに強制配置する場合、宣言をプロジェクト全体で一致させる必要があります。一致していない場合は、リンカでエラーが発生する場合があります。
- コンパイラとリンカの初期設定閾値は 8 バイトです。サイズが 8 バイト以下のオブジェクトは、小データと見なされます。この閾値は、コマンドラインで-G オプションを使用 すれば、変更できます。たとえば、-G16 を使用すると、小データセクショ ンに配置できるオブジェクトを増やせます。
- ビルド処理の各過程で、異なる-G オプションの値を使用すると、リンカエラーが 発生する場合があります。共通オブジェクトに-G オプションを使って小データとするかどうかを決定している場所をリンクした場合も、エラーが起こる場合があります。すべてのライブラリは、デフォルト設定でビルドされています。
- ライブラリのグローバル変数にリンクを張る際に問題が生じた場合は、その変数を extern として、適切なセクション内で宣言してみて下さい。

トラブルシューティング

「未定義シンボルへの参照」エラー

ngcld リンカは、コマンドラインで指定されたライブラリの命令に依存しています。ライブ ラリがコマンドラインでの参照に先だって、シンボルを指定された他のライブラリで参照した場合には、「未定義シンボル、、、」エラーが表示されます。

この命令依存を削除するには、リンカコマンドラインの -start-group <libs> -end-group を指定してください。ライブラリのグルーピングにはビルド時間が増加することに、ご注意ください。この問題を解決するもっとも有効な方法は、ライブラリが、コマンドラインでの参照に先だって、ライブラリのリストからシンボルを参照しないように、ライブラリに命令することです。

第 8 章: ビルドの最適化

はじめに

この章では、Nintendo GameCube で ProDG を使用する際の、ビルディングと実行コードを最適化する方法の概要をご説明いたします。さらに、SN Systems の Web サイトのテクニカルサポートセクションでの、さらに掘り下げた最適化に関する詳細の説明もいたします。(2 ページの「テクニカル サポートとアップグレード」を参照)

コード サイズ、ビルド速度、コード実行速度の最適化には、常にトレードオフがあります。このため、最適化の対象とその結果を比較検討し、最も最適化が必要な部分に焦点を当てることが重要になります。すべてを最適化することも可能ですが、焦点を絞った最適化の方が、良い結果に結びつきます。

ELF のサイズ最適化

ELF ファイルのサイズを最適化してメモリ使用量を節約するには、プログラムのコード、データ、デバッグ情報の量を減らす必要があります。

小さい ELF にはメモリ使用量削減以外に、ビルド時間の短縮、およびキャッシュ使用効率の向上による実行時のパフォーマンス向上が期待できます。最適化によるこれらの利点を考慮すると、ELF のサイズは、可能な限り小さくする必要があります。以下のセクションでは、サイズ縮小の方法を説明します。

コンパイラ オプション

以下の表は、ELF ファイルのサイズ縮小に役立つコンパイラ スイッチの一覧です。

オプション	機能
-fbyte-bool	C++ の 'ブール' タイプを、デフォルトの 4 バイト長から 1 バイト長に変更します。**メモ**:このスイッチは、プロジェクト全体を通して使用する必要があります (各コンパイル ユニットで異なるブール サイズを使用しない)。
-fdefault-single-float	定数のデフォルトに、ダブルではなくシングルを強制的に使用します。このため、「f」を指定する必要がなくなります。
-fforce-link-once	デバッグ データの生成時でも、テンプレート インスタンスおよび導入されたインライン メソッドを、リンク ワンス セクションに強制的に配置します (デフォルトではデバッグ時にリンクワンス セクションを使用しない)。このため、繰り返されるテンプレート インスタンスと導入されたインライン メソッド

	の、リンク時の自動削除が可能になります。
-fno-exceptions	C++ の例外を処理するコードを生成しません。
-fno-implement-inlines	インライン化された関数の非インライン インスタンスを生 成しません。詳細は、67 ページの 「重複する非インライン関数の生成防止」を参照してください。
-fno-rtti	C++ の RTTI (ランタイム タイプ情報) を処理するコードを生成しません。
-fno-static-dtors	オブジェクト ファイルからグローバル デストラクタ コード を削除します。詳細は、67 ページの「グローバル デストラクタの削除」を参照してください。
-fvtable-thunks	仮想ディスパッチテーブル('vtable')を実現するために 'thunks'を使います。

C++ コード中に RTTI や例外を使用しているエリアがある場合は、そのエリアの 構造を変更してゲームからの削除を可能にするのもひとつの方法です。

リンカ オプション

以下のリンカ オプションは、生成する ELF ファイルのサイズ縮小に役立ちます。

オプション	機能
-s または -strip-all	ELF 生成時に、すべてのシンボル情報を削除し ます。詳細は、67 ページの「シンボル情報の削除」 を参照してください。
-S または -strip-debug	生成された ELF からデバッグ情報を削除します。 シンボル情報は削除されません。
-s-lib	ライブラリからデバッグ情報を削除します。オブジェクト ファイルからは削除しません。詳細は、67 ページの 「重複する非インライン関数の生成防止」を参照してください。
-sn-no-dtors	グローバル デストラクタをリンクしません。詳細は、67 ページの「グローバル デストラクタの削除」 を参照してください。
-strip-unused	使用されないコードを ELF から削除します。 この機能では、重複するテンプレート メソッドのインスタンス が削除される (非デバッグ ビルドでは自動的に削除) ため、多くのテンプレートが使用されているコードのデバッグ ビルドに便利です。
-strip-unused-data	使用されないコードとデータを ELF から削除します。「-strip-unused」とは異なり、使用されないデー タと同時に、削除されるデータ構造のみから参照されるコードも削除されます。

シンボル情報の削除

「-s」オプションまたは「-strip-all」オプションを使用すると、すべてのシンボル 情報が削除されて ELF のサイズが小さくなります。

不必要なデバッグ情報の削除

デバッグ情報は、デバッグの対象となるファイルに対してのみ作成してください。デバッグするファイルにはコンパイラ スイッチ「-g」、デバッグしないファイルには「-g0」を使用 してください。

メモ: Visual Studio との統合機能を使用している場合は、「-g」オプションをファイル毎に設定することはできません。しかし、プロジェクトをライブラリに分割することにより、デバッグしないプロジェクト ライブラリのデバッグ情報生成をオフに設定することは可能です。この場合、デバッグをしないコードは、メイン プロジェクトからライブラリに移動してください。

デバッグするすべてのコードがオブジェクトとしてリンクされている場合は、リンカ オプション「-s-lib」を使用することにより、リンクする任意のライブラリからデバッグ情報を削除することができます。

重複する非インライン関数の生成防止

多くの関数をインライン化している場合は、コンパイラ スイッチ「-fno-implement-inlines」を使用することによって、重複する非インライン関数の生成を回避することができます。しかし、これらの非インライン関数がコードからコールされる場合は、最低 1 つのコンピレーション モジュールにこれら関数のインスタンスが含まれている必要があります。含まれていない場合は、リンク タイム エラーが発生します。

このオプションを使用することによって ELF のサイズは縮小されますが、ビルド時間が長くなる可能性があります。

GNU コンパイラでは、非インライン インスタンスを生成しないための言語拡張機能が使用されます。ヘッダで関数が「external inline」と宣言されている場合、非インライン インスタンスは生成されません。

グローバル デストラクタの削除

グローバル デストラクタ コードは、クラスのグローバル インスタンス用のデストラクタをコールします。プログラムが終了することない場合（コンソールをオフにする）で、メモリの開放の必要がない場合などのために、コンパイラとリンカのオプションには、グローバル デストラクタ コードを削除するためのものが用意されています。グローバル デストラクタの削除は、不要コードの削除と併用することにより、特に効果が現れます。このオプションは、DLL には使用しないでください (DLL はアンロードされた際にクラスのグローバル インスタンスが開放されるため)。

コンパイラ オプション「-fno-static-dtors」は、オブジェクト ファイルからグローバル デストラクタ コードを削除し、リンカ オプション「-sn-no-dtors」は、グローバル デストラクタがリンクされるのを抑止します。またこのリンカ オプションを使用し、コンパイラ オプション「-fno -static-dtors」を使用せずにビルドされたライブラリ コードからデストラクタを削除することも可能です。最良の結果を得るためには、両方のオプションを使用してください。

実行速度向上のための最適化

コードを最適化して実行速度を向上するには、CPU 使用時間を最も多く必要とするセクションを集中的に探し出す必要があります。ProDG Debugger for Nintendo GameCube に搭載されている基本的な Gekko プロファイラを使用することもできますが、高度な Tuner for Nintendo GameCube を使用することにより、最も最適化が必要な部分以外のコード エリアを発見することが可能です。

ELF サイズの最適化を行った場合は、キャッシュ使用率の効率化により、それ自体で実行時のパフォーマンス向上に繋がります。詳細は、65 ページの「ELF のサイズ最適化」を参照してください。

コンパイラの最適化スイッチの使用方法

コンパイラには多くの最適化スイッチが用意されています。「-Ox」スイッチを使用した定義済みの最適化機能も用意されているため、シンプルな用途に使用することもできます。このスイッチでは、0 から 3 までの任意数字の接尾辞を指定することにより、「-O0」から「-O3」までの 4 つの値を使用することができます。一般的に大きい数字は、多くのオプションが使用され、コードの実行速度が最も向上することを表しますが、すべてのファイルに「-O3」フラグを使用した場合は、「-O2」を使用してコンパイルされたゲームよりパフォーマンスが低いことがあるため、注意が必要です。

「-O2」と「-O3」との大きな違いは、「-O3」では自動インライン化が使用され、関数の宣言にインラインが含まれていない場合でも、コンパイラが関数をインライン化する点です。これにより、サイズの大きいコードが生成される可能性が高くなります。サイズが大きいコードでは、キャッシュ ミスが増えてキャッシュの効率が低下するため、全体的な実行速度も低下します。このため、すべてのファイルに対して「-O2」を使用すると、「-O3」を使用した場合と比較して、コードの実行速度が向上します。

「-O3」スイッチは、一部のキーとなるモジュールに限定して使用することによってパフォーマンスの向上に結び付けることができます。これらのモジュールは、ProDG Debugger for Nintendo GameCube の基本 Gekko プロファイラを使用して発見することができます。この場合は、モジュール内の特定の関数を、インライン宣言することによってインラインの対象とし、次に「-O2」スイッチを使用します。

ビルド速度の最適化

ビルド時間短縮の最も有効な方法は、ELF のサイズを縮小してコンパイルするコード (特にデバッグ コード) を少なくすることです。サイズの縮小に関する詳細は、65 ページの「ELF のサイズ最適化」を参照してください。以下のセクションでは、ビルド時間短縮に関するその他の方法について説明します。

不必要なインクルード ファイルの排除

ビルド時間は、ソース プロジェクトを慎重にプランすることによって短縮することができます。この際、最も重要になるのはインクルード ファイルの使用方法です。コンパイルするモジュールに最小限のヘッダのみが含まれる構造のプロジェクトでは、コンパイル時間、リンク時間、およびデバッグ ビルドではデバッガの読み込み時間が大幅に短縮されます。コンパイラでは、ヘッダ ファイルに含まれるすべてのタイプに対してデバッグ情報が生成されます。

モジュールに不必要なヘッダが多く含まれている場合は、それに応じてデバッグ データが肥大化し、リンク時間およびデバッガへのプログラム読み込み時間も増加します。

C++ で記述されたテンプレートをヘッダに使用した場合も、デバッグとリリースの両方のビルドにおいて問題が発生する可能性があります。コンパイルするモジュールに多くの相互依存テンプレート コードが含まれている場合、コンパイル毎にオーバーヘッドが発生してテンプレート メソッドのインスタンスが生成されます。ビルド速度を向上するには、テンプレートを含むヘッダの使用を最小限に抑えてください。

また、マップまたはデバッグ ログなどのリンカ オプションを使用した場合も、ビルド時間が長くなります。

システム リソースの最大化

- ハードウェアやソフトウェアに変更を加える場合は、常にシステムのバックアップを行ってください。

高速なビルドを行うには、メモリの使用を実際にインストールされている RAM 以内に抑えます。メモリ使用量がこれを超える場合は、RAM を増設することをお勧めします。

Windows 2000 など、最近のバージョンの Windows を使用している場合は、以下の方法でメモリ使用量を確認することができます。

1. ビルド中に Task Manager を実行 (Ctrl+Alt+Del を押して [**Task Manager**] を選択) します。

2. [**Performance**] タブをクリックしてメモリ使用量を確認します。

以下のように、他の使用されていないアプリケーションやサーバーを終了してビルド時間を短縮することもできます。

- すべてのファイル ブラウザおよび Web ブラウザのウィンドウを閉じる。ファイルの更新に合わせて表示が更新されるファイル ブラウザ ウィンドウが開いていると、ビルド速度が極端に低下します。

- 電子メール クライアントと Personal Web Server を閉じる。

- ビルドに必要のないアプリケーションを閉じる。

- ドライブのネットワーク共有を無効にする。

- ウイルス チェックを無効にする。

メモ: 変更を行った場合、特にウイルス チェックを無効にした場合などは、その変更が許容範囲内のものであることをシステム管理者に確認してください。

ビルド時間は、ディスクのハウスキーピングの状態によっても左右されるため、残り容量やフラグメンテーションにも注意する必要があります。

- 古いプロジェクトのオブジェクト□ファイルなど、不必要なファイルは可能な限り削除する。検索機能でファイルの種類に「Intermediate files」を指定すると効率的です。

- 特定サイズ以上のファイルや特定日時以前などを指定してファイルを検索し、必要に応じて削除する。

- 使用していない一時フォルダ内のファイルを削除する。一時フォルダは複数存在する場合があるため、フォルダ名を「temp」などと指定して検索すると効率的です。

- ハードディスクの空きスペースを 25% 以上確保する。

不必要なファイルを整理した後は、デフラグを実行してください。デフラグ処理には時間がかかるため、コンピュータを使用していない夜中などに実行することをお勧めします。また、デフラグを実行する前にはコンピュータを再起動し、デフラグ終了後にもコンピュータを再起動してください。

テクニカルサポート

ProDG の使用に関する問題は、以下の SN Systems Web サイトのサポートにアクセスしてください (2 ページの「テクニカル サポートとアップグレード」を参照)。

テクニカル サポートにお問い合わせになる前に、数多くのドキュメントや ProDG に関連する FAQ が収録されている、テクニカル ライブラリを検索することをお勧めします。

SN Systems へのフィードバック

ビルドに時間がかかりすぎる場合は、当社までご連絡ください。また同時に、マップ生成の排除、および関数やデバッグ情報の削除などもお試しください。これによってビルド時間が大幅に向上した場合は、当社へのレポートにその内容を記載してください。お問い合わせの際は、可能な限りの事例をご提供ください。

以下は、当社での問題の診断に必要となる情報です。

- PC のハードウェア環境および OS に関する詳細な情報

- ご使用の ProDG ツールのバージョン (「-v」オプション)

- Visual Studio の使用有無、特殊オプションの使用有無

- デバッグ情報/マップファイル生成/削除機能の使用有無

Web サイトのテクニカル ライブラリで、問題の解決策が見つからない場合は、SN Systems のサポートチームに電子メールでお問い合わせください。(2 ページの「テクニカル サポートとアップグレード」を参照)。

第 9 章: 再配置可能モジュール

再配置可能モジュールをビルドする

ニンテンドーゲームキューブ OS で再配置可能モジュールをビルドできるようになりました。再配置可能モジュールには.rel ファイル拡張子があり、ディスクからメモリにロード、OSLink()などの OS コールを使って、ランタイムに動的に現行プログラムリンクできます。

各々のモジュールファイルは、任天堂の makerel を SN Systems に改良した snmakerel.exe を用い、部分的にリンクされた.ELF ファイル(.plf 拡張子付き)でビルドされます。

snmakerel は部分的にリンクされた.ELF ファイルからコードとデータを解凍するだけでなく、モジュールの再配置に必要な情報も抽出、ロード可能な他のモジュールへのリンクもします。再配置可能モジュールファイルはできるだけ小さいほうがよいため、再配置情報はシンボル名なしでストアされます。また、全ての再配置可能モジュールは同時に作成される必要があります。

加えて、モジュールが関数をコールできるようにするため、再配置可能モジュールの作成時にプログラムの静的パートが必要になります。

- 一般的に C コードはオブジェクトファイルにコンパイルされた後、静的.ELF ファイルにまとめてリンクされます。.ELF ファイルに含まれるコードはゲームのコンソールの決められたアドレスにロードされ、そこから実行されます。複数のオブジェクトをリンクしてひとつの新しい大きなオブジェクトファイルを作る場合には部分リンクが作成されます。このオブジェクトファイルは静的プログラムを作成するために後で行われるリンクで使用されます。任天堂の再配置可能モジュールはこの部分リンクプロセスを利用して作成されますが、静的プログラムを作成するために再リンクされるのではなく、リンクは実行時に行われます。部分リンクされた.ELF ファイルに含まれているシンボル情報が多すぎるため、部分リンクされた.ELF を.rel 形式に変換する snmakerel が必要になります。リンカに部分リンクを行わせる時にはリンカコマンドラインで–r を使用してください。

- 再配置モジュールをビルドしリンクするときは小データ領域の最適化が使用できないことに注意してください(62ページの「小データ領域のサポート」を参照してください)。小データ領域を使わないようにするためには、ビルド、リンク時にコマンドラインで–G0 を使用してください。

- オブジェクトファイルには共通データが含まれている場合があります。これらはメモリに割り当てられていない初期化されていないデータです。リンカは通常、最終的な静的プログラムを作成するときに割り当てを行います。部分リンクの場合には共通なデータ割り当ては行われませんが、部分リンクファイルはリンカによって

静的プログラムにはリンクされませんので、こういった場合にはすべての共通データは割り当てられなければいけません。リンカコマンドラインで–d を指定して、部分リンクであっても共通データを割り当てるように指示してください。

- snmakerel は部分リンクされた.ELF ファイルでたかだか数個のセクションしか利用されないと想定しています。通常 1 つのオブジェクトには多数のセクションタイプが含まれます。オブジェクトから部分リンクされたファイルを作成する場合にはリンカスクリプトの preplf.ld を利用してください。これによりオブジェクトによって作成された様々なセクションが集められ、makerel が必要とする正しいセクションに置かれます。preplf.ld ファイルもグローバルコンストラクタとデコンストラクタのすべてのテーブルを集めます。また、LONG(0)コマンドを利用して各リストの最後に終了を示す 0 が置かれます。すべてのセクションはアドレス 0 にリンクされている必要があります。実 RAM アドレスへのオフセットは後で与えられます。これを実現するためにリンカスクリプトは OVERLAY コマンドを利用します。これは実際にオーバーレイを生成するものではなく、すべてのセクションを要求に応じてアドレス 0 にリンクする既存のリンカスクリプト手法です。オブジェクトから部分リンクされたファイルを作成する際に、リンカコマンドラインで–T preplf.ld を使用します。

- 部分リンクの 2 度目には短縮されたリンカスクリプトが使われます。終了の 0 バイトを終了リストに追加する必要はありません。また、すでに実施済みですのでセクションを並べ替えする必要もありません。削除を有効にして preplf ファイルを部分リンクする際にリンカコマンドラインで-T plf.ld を利用してください。

再配置モジュールをビルドする

1. オブジェクトファイルを-r オプションを使ってリンクしてください。次の例のように、ProDG コンパイラドライバを利用する場合には引数をリンカに渡すために ngccc –Wl、-r を使用し、ProDG リンカを直接利用する場合には ngcld –r を使用してください。

   ```
   ngccc -Wl,-r,-G0,-d,-T,preplf.ld <OBJECT FILES>
   ```

   または

   ```
   ngcld -r -G0 -d -T preplf.ld <OBJECT FILES>
   ```

   このプロセスからの出力は *pre partially linked ELF file* (PREPLF ファイル)で、ファイル名の拡張子は `.preplf` です。

   ビルドしようとしている再配置可能モジュールごとに別の `.preplf` をビルドする必要があります。

   事前に部分リンクされたファイルは削除可能なシンボルを含んでいる場合があります。すべてのモジュールがビルドされるまでは必要となるかもしれないすべてのシンボルのリストを作成することができます。いちどリストが作成されると削除され、部分リンクされた ELF ファイルを生成するために再び部分リンクすることができます。

2. snmakerel を利用して *keep lis*t をビルドします。このためには最終ゲームで利用したいすべての再配置可能ライブラリのための.preplf ファイルが先にビルドされている必要があります。

```
snmakerel a.preplf b.preplf ...
```

keep リストのファイル名はデフォルトでは import.lst です。

3. 次に各.preplf ファイルでは生成された keep リストと *partially linked ELF files* (PLF ファイル)を利用して関数削除を行うことができます。また、plfkeep.txt にリストされたモジュールへのエントリポイントを保持する必要があります。これによりコマンドラインの–keep plfkeep.txt を含むエントリポイントが削除されるのを防ぎます。リンカコマンドラインの–strip-unused スイッチは次のように使われていない関数を削除するために利用されます。

```
ngcld -keep import.lst a.preplf -keep plfkeep.txt -
strip-unused -T plf.ld <OTHER OPTIONS>
ngcld -keep import.lst b.preplf -keep plfkeep.txt -
strip-unused -T plf.ld <OTHER OPTIONS>
etc.
```

この段階での出力は.preplf ファイルひとつひとつに対応した.plf ファイルです。

4. この段階でゲームの静的部分をビルドすることができます。再配置可能モジュールで必要とされるライブラリ関数はリンクされます。

```
ngcld <OBJECT FILES> -keep import.lst -sn-force-
section .sn_relsymbols
```

5. .plf ファイルを.rel の拡張子がついた再配置可能モジュールに変換することができます。また static.elf にデバッグ情報が追加されます。これによりゲームをデバッグするときにデバッガは.plf ファイルを見つけることができます。

```
snmakerel a.plf b.plf ... static.elf
```

この手順での出力は一連の.rel ファイル(a.rel、b.rel 等)と修正された static.elf になります。

[このページは意図的に空白にされています]

第 10 章: ユーティリティ

blankelf - blank ELF ユーティリティ

BLANKELF の目的は ELF ファイルの中のコードセクションをゼロで埋めることです。弊社にデバッグあるいはシンボルデータに問題のある ELF ファイルを送付してくる際、完全に動作する ELF ファイルを公表したくない場合に使用されます。BLANKELF を使用してすべてのコードをゼロにし、ELF を意味を持たないものにすることができます。しかしながら、実行不可能なコードであっても SN Systems サポートチームは ELF をロードしてデバッグあるいはシンボルデータに問題があるかどうかを調べることが可能です。

メモ: BLANKELF ユーティリティは ELF ファイルの所定の位置でコードを上書きします。ファイルのコピーを作成するものではありません!

ELF file を空白にするには、

1. BLANKELF.EXE プログラムを開発を行っているディレクトリにコピーする。
2. ELF ファイルをどこか安全な場所にコピーする – 重要!
3. コマンドプロンプトで BLANKELF <ファイル名>とタイプする。<ファイル名>は ELF の名前です。
4. これで ELF ファイルはデバッグとシンボルの情報だけを表示します。

make

make.exe は標準のビルドユーティリティです。makefile で特定されている入力/出力の従属関係を解釈します。

ProDG に備えられた make.exe の初期設定では大文字と小文字を区別します。例えば makefile の中で特定されるファイル名は開発中の PC でのファイル名と大文字、小文字が一致していなくてはなりません。しかしながら SN Systems のウェブサイトから大文字、小文字を区別しないバージョンの make ユーティリティの v3.78.1 をダウンロードすることができます。これにより、makefile で特定されたファイル名と大文字、小文字が一致しないファイル名で make を使用することができるようになります。

Makefiles と make ユーティリティに関しての詳細については**www.gnu.org**で GNU のドキュメンテーションをご覧ください。

snarl - SN アーカイヴライブラリアン

SN ARchive **L**ibrarian（SN アーカイヴライブラリアン、snarl）は GNU ar 使用に準拠します。

コマンドライン、もしくは MRI スクリプトから `snarl` もしくは `ar` をコントロ ールする方法についての情報は以下のウェブサイトでご覧いただけます。**www.gnu.org**.

以下のセクションは snarl の追加機能について説明します。

ライブラリの読み込み

最新版はさらに Metrowerks Codewarrior フォーマットのライブラリも読み込み可能となりました。ライブラリ所収の情報（コンテンツの表示など）を表示したり、Metrowerks フォーマットの保存も可能ですが、操作コマンドを実行すると、ライブラリは弊社ツールに完全互換の GNU 基準にコンバートされます。

コンテンツを変更せずにフォーマットを GNU フォーマットにコンバートしたい場合は、そのままシンボル表オプション('s')を使用してリビルドしてください。例：

```
snarl s mwformat.a
```

クロスプラットフォーム

snarl は、ほとんど全てのプラットフォームのライブラリの読み込み、作成が可能です（Win32 を除く）。ニンテンドーゲームキューブ、ゲームボーイアドバンス、ソニー PlayStation 2 各々のプラットフォームでテスト済みです。

異なるプラットフォーム用にビルドされたオブジェクトファイルを含むライブラリの作成も可能です。例えば、ライブラリ sky.a は ngcsky.o、agbsky.o、および ps2sky.o のオブジェクトを内包できます。また、クロスプラットフォームのゲームライブラリの監理を容易にするため、NGC_getskycoords(), AGB_getskycoords(), and PS2_getskycoords()のような関数も漸次内包できます。

コマンドライン書式

`snarl` は作成、修正、アーカイヴからの抽出などを可能にします。この場合、アーカイヴとは通常ライブラリを意味します。例えば `libc.a`.の場合、

使用法：

`snarl [-]`キー`[mod [relpos] [count]]` アーカイヴ `[file(s)...] [symbol(s)...]`

`snarl -M [<mri-script>]` snarl -M? を使用して MRI スクリプトコマンドを表示

`snarl archive @responsefile` 簡単な応答ファイルを使用

`snarl -E` snarl と.sns ファイルを結合させる

`snarl --help` コマンドリストの印刷

キーは以下のいずれか、

d[NWT]	ファイルをアーカイヴから消去
m[abNWT]	ファイルをアーカイヴから移動
p[N]	アーカイヴのファイルを印刷
q[cfSWT]	アーカイヴにファイルをすばやく追加
r[abucfSWT]	アーカイヴに現存もしくは新規ファイルを挿入
s[WT]	シンボル表 (ranlib) をリビルド（デフォルトで実行）
t	アーカイヴのファイル用コンテンツ表を印刷
h[N]	アーカイヴのファイルの mod 時を現在時刻にセット
w[l]	アーカイヴシンボル表を表示
F[lN]	ファイル用シンボル表を全て表示
x[oCN]	アーカイヴからファイルを抽出

もしくは以下のシンボル操作コマンドである必要があります。

G	シンボルをグローバルにする
W	シンボルを非重要項目にする
L	シンボルをローカルにする

使用可能なモジュールは、

A	アーカイヴメンバー[relpos]後にファイルを追加
B	アーカイヴメンバー[relpos]前にファイルを追加
C	アーカイヴ作成が必要な時に警告しない
C	解凍ファイルの既存ファイルオーバーライトを許可しない
F	8.3 ファーマットに挿入ファイル名をトランケート
L	C++ シンボル名をデマングル（demangle.dll が使用可能な時）
N	アーカイヴの同じファイル名[count]インスタンスを定義
O	元の日付けを保存
S	シンボル表のビルドを抑制する（デフォルトでビルドされるため）
T	複数回シンボルが定義された時警告
U	ファイルが新しい時のみアーカイヴメンバーを置換
V	Verbose モード
V	バージョンを表示

Verbose モード

Snarl のほとんどのキー文字に「v」を追加することによって verbose モードに入ります。もっとも有用な方法として、目次('t')コマンドの印刷があげられます。これにより拡張情報にライブラリのコンテンツが強制的に表示されます。

アーカイヴファイルの印刷

snarl は、いかなるファイルタイプのアーカイヴ作成に使用できます（リンク時ごとにオブジェクトファイルのみアーカイヴシンボル表に含まれることは言うまでもありません）。例えばバージョン史を含むテキストファイルは簡単にライブラリにストアできます。

これは**アーカイヴファイルの印刷**('p')コマンドを使用して閲覧できます。例えば、

```
snarl p test.lib versions.txt
```

オブジェクトファイルの印刷

その他のライブラリアーカイバの扱わない事項として、オブジェクトファイルが指定されている時は、**アーカイヴファイルの印刷**('p')は 正常に動作しない、ということが上げられます。それでも snarl はテキストベースファイルは通常通り表示しますが、オブジェクトファイルが無効になっているときは、自動的に 16 進法フォーマットでダンプするようスイッチします。これは stdout に送られるので、'>' DOS リダイレクトコマンドを使えば、ファイルを移動できます。例えば、

```
snarl p test.lib obj1.o obj2.o > out.txt
```

シンボル表の表示

キー文字'w'は stdout にアーカイヴシンボル表をダンプします。 これはどの シンボルがリンカに閲覧可で、ライブラリ内のどのオブジェクトファイルに属しているのかを示します。

バージョン 1.3.3.95 以降は C++シンボル名のデマングルをサポートしています。

キー文字'F'はオブジェクトファイルの全てをシンボル表から stdout にダンプします （バージョン 1.4.2.9 以降）。これにより、ライブラリのシンボル全てがタイプやスコープと同様、閲覧可能になります。かなりの量になりますので、ファイルを配列することをお勧めします。

```
snarl F test.lib > out.txt
```

高速追加

ライブラリを段階に分けてビルドする場合、（すなわち全てのオブジェクトファイルを一度に追加するのではなく）「高速追加」機能をご利用いただけます。簡便追加 ('q')が指定され、シンボル表のビルドが抑制('S')されている場合、この機能は有効となります。ただし、大規模のライブラリ（20MB 以上）で作業中のみ有効です。小規模ライブラリの時には、それほどの効果がないためです。

新規ファイルを現存データにロードすることなくライブラリに追加することで機能します。ライブラリを機能させるため、シンボル表は一度任意の操作完了後にビルドしておく必要があります。

以下は、4つのオブジェクトファイルをすばやく大規模ライブラリに追加する方法を示した例です。

通常作業:

```
snarl q test.lib obj1.o obj2.o
...
snarl q test.lib obj3.o
...
snarl q test.lib obj4.o
```

高速追加の場合:

```
snarl qS test.lib obj1.o obj2.o
...
snarl qS test.lib obj3.o
...
snarl qS test.lib obj4.o
snarl s test.lib            // シンボル表をリビルド
```

一つめの例では各々の追加時に費やされる時間は *N* です(*N* は現存ライブラリのサイズに応じて生じる時間の遅延です)。合計、3*N* 秒かかることになります。

二つめの例では各々の追加時にかかる時間は≈ 0 秒、シンボル表の最終ビルドにかかる時間が *N* 秒となっています。かかる時間 *N* 秒です。

> **注:** もし オブジェクトファイルのファイル名を 15 文字以上で追加したい時は一つめの追加はキャンセルされます。拡張ファイル名セクションのリビルドが必要なためです。解決方法として、f 文字を使えば、ファイル名を 8.3 DOS フォーマットに短縮します。snarl qSf test.lib のファイル名は 15 字以上です。

複数回定義されたシンボルの警告

'T'は複数回定義されたシンボルが現れた際に警告を出し、既存のキーコマンドのいずれに追加できます。例えば、高速追加を実行した際、複数回定義されたシンボルの使用を警告します。

```
snarl qT test.lib obj1.o obj2.o obj3.o obj4.o obj5.o
```

ライブラリに手を入れることなく、単に複数回定義されたシンボルをリストのみしたい場合は's'をキーコマンドとして指定、'T'を修飾語句として指定して下さい。アーカイヴシンボル表をリビルドし、ライブラリコンテンツは変えることなく、警告を出します。例:

```
snarl sT test.lib
```

Visual Studio 統合を使用してライブラリをビルドすればこの機能は自動的に作動されます。警告は Visual Studio のビルドウィンドウに表示されます。

シンボル操作コマンド

3つのシンボル操作コマンドがあります。ライブラリの内部でシンボルプロパティを修正する時に使う、"グローバルにする" (G)、"非重要項目にする"(W)、および"ローカルにする"(L)の3つです。(バージョン 1.4.2.9 以降)

次の例では、test.lib のシンボルを非重要シンボルに変換します。

```
snarl W test.lib sym1
```

複数のシンボルを一つのコマンドラインに指定することもできます。

```
snarl G test.1 sym1 sym2 sym3
```

注:コマンドが個別に使われた時は、ライブラリ内のシンボルを含有するオブジェクトファイルのシンボルプロパティを変更します。アーカイヴシンボル表は更新されません。すなわち、ローカルシンボルをグローバルとした場合、アーカイヴシンボル表をリビルドするまで、リンカには不詳となります。もちろん、`'s'`引数を使えば、同時に指定できます。

```
snarl Gs test.1 sym1 sym2 sym3
```

応答ファイルスクリプティング

`Snarl` はシンプルな応答ファイルをサポートしています。例えば、

```
snarl lib.a @response.txt
```

ここで `response.txt` はフォーマットのテキストファイルです。

```
object1.o
object2.o
object3.o
object4.o
[etc.]
```

応答ファイルは同じ名称のアーカイヴは自動的に削除し、新規ビルドし直しますのでご注意下さい(追加は不可能です)。より複雑な処理には MRI スクリプティングをご使用ください(詳しくは80ページの「MRI スクリプティング」を参照)。

MRI スクリプティング

コマンドラインで snarl -M を使用すると、snarl を簡単なコマンド言語として操作することができます。 Snarl は'SNARL >'というプロンプトを表示して入力ができるようになります。snarl コマンド言語はコマンドラインのオプションよりも操作できる種類が少ないものの、MRI"librarian"形式でスクリプトを作成する手間を省きます。

MRI スクリプトコマンドの形式は以下の通りです。

- 1 行に1コマンド
- 大文字小文字の認識無し
- '*'か';'以降の文字はコメント

- コマンドの引数はカンマかスペースで分けられる
- '+'は継続を表し、このマークがついている行の文字は、現在のコマンドと同等に扱われる

以下の表は MRI スクリプトで使用できるコマンド一覧です。

コマンド	機能
ADDLIB archive ADDLIB archive (module, module, ... module)	現在のアーカイヴに指定されたアーカイヴの内容全て(または、指定されているアーカイヴ内の各モジュール)を追加。このコマンドの前に OPEN か CREATE コマンドが必要。
ADDMOD member, member, ... member	各名前付メンバーを現在のアーカイヴ中のモジュールとして追加。このコマンドの前に OPEN か CREATE コマンドが必要。
CLEAR	現在のアーカイヴの内容を消去し、最後の SAVE 以降の操作の結果をキャンセルする。現在のアーカイヴに指定がなくても実行はされる(何も起こらない)。
CREATE archive	アーカイヴを作り、それを現在のアーカイヴとする(他のいろいろなコマンドで必要となる)。新しいアーカイヴは一時的な名前で作られ、SAVE を行なうまで実際にアーカイブとして書き込まれない。既にあるアーカイヴを上書きすることができるが、名前のついたファイルの内容は SAVE を行なうまでは破壊されない。
DELETE module, module, ... module	現在のアーカイヴからリストされたモジュールを削除する。これは'snarl d archive module ... module'と同じ効果。このコマンドの前に OPEN か CREATE コマンドが必要。
DIRECTORY archive (module, ... module) DIRECTORY archive (module, ... module) outputfile	アーカイヴ中にある名前付モジュールをリストする。別のコマンドである VERBOSE は出力の形式を指定する。Verbose 出力がオフの場合には、出力は'snarl t archive module...'と同じ。オンの場合には、リストは'snarl tv archive module...'と同じ。出力は普通は標準出力に出されるが、出力ファイルを最後のパラメータで指定した場合には、snarl はそのファイルに出力する。
END	正常終了を示す exit code 0 で snarl から出る。このコマンドは出力ファイルをセーブしない。最後の SAVE コマンド以降に現在のアーカイヴに変更を加えている場合には、その変更は失われる。
EXTRACT module, module, ... module	名前付モジュールを現在のアーカイヴから取り出し、それらを現在のディレクトリにそれぞれファイルとして書き出する。これは'snarl x archive module...'と同じ。このコマンドの前に OPEN か CREATE コマンドが必

	要。
GLOBAL symbol, symbol, ... symbol	現行アーカイヴからリストされた各々のシンボルをグローバルにする。'snarl G archive symbol... symbol 'と同様。「開く」、もしくは「作成」に先立って使用のこと。
LIST	VERBOSE の設定にかかわらず、現在のアーカイヴの内容を"verbose"形式で表示する。この結果は'snarl tv archive'と同じ。このコマンドの前に OPEN か CREATE コマンドが必要。
LOCAL symbol, symbol, ... symbol	現行アーカイヴからリストされた各々のシンボルをローカルにする。'snarl L archive symbol... symbol'と同様。「開く」、もしくは「作成」に先立って使用のこと。
OPEN archive	既存のアーカイヴを現在のアーカイヴとして使うためにオープンする（他のコマンドで必要な場合が多くある）。これに続いて投入するコマンドの結果は SAVE コマンドが使われるまではアーカイヴに反映されない。
REPLACE module, module, ... module	現在のアーカイヴにおいて、既存のモジュール（REPLACE コマンドで指定されている）を現在のワーキングディレクトリ中のファイルに換える。このコマンドをエラーなしに実行するためには、そのファイルと現在のアーカイヴにあるモジュールの両方が存在するものでなければならない。このコマンドの前に OPEN か CREATE コマンドが必要。
SAVE	現在のアーカイヴに適用された変更を反映させ、最後の CREATE か OPEN コマンドで指定されたファイル名でセーブする。このコマンドの前に OPEN か CREATE コマンドが必要。
VERBOSE	DIRECTORY の出力を決める内部フラグを切り替える。このフラグがオンだと、DIRECTORY の出力は 'snarl tv 'の出力と同一になる。
WEAK symbol, symbol, ... symbol	現行アーカイヴからリストされた各々のシンボルを非重要項目にする。'snarl W archive symbol... symbol'と同様。「開く」、もしくは「作成」に先立って使用のこと。
$(ENV)	環境変数マクロ拡張。このフォーマットのいかなるマクロも実行時に指定の環境変数の値に拡張されます。例: open $(LIB_DIR)\lib.a

MRI スクリプトをファイルから実行する

MRI スクリプトをコマンドプロンプトから実行する

`snarl -M [<mri-script>]` MRI スクリプトコマンドのリストを表示するには snarl -M?を使用してください。

この場合、snarl は MRI コマンドを<mri-script>で指定される MRI スクリプトファイルから取り出します。snarl スクリプトのファイル名のデフォルト拡張子は .sns です。

Windows Explorer から MRI スクリプトを実行する方法

1. snarl と.sns (snarl スクリプト) 機能拡張をコマンド'snarl -E'を使用して結合させる
2. .sns ファイル名機能拡張で全てのスクリプトをプレーンテキスト形式で保存
3. 保存したファイルをダブルクリックして自動実行

[このページは意図的に空白にされています]

第 11 章: Gekko プロファイリング用ビルド

ゲームの中の Gekko プロファイリングをサポートするには

Gekko プロファイルペイン使用に際しての手順は、以下の通りです。

1. 使用中のコードにヘッダファイル libsn.h をインクルード
2. プロファイリング開始のため、プログラムをコールするよう、関数を追加
3. libsn.a とリンク

これだけで済みます。ProDG デバッガのプロファイル機能が残りの作業をします。お使いの GameCube プログラムは、これで最高速で実行されます。

Gekko プロファイラ API

プロファイラ データコレクションを可能にする libsn.a に、簡単な関数があります。

```
int snProfInit(UINT32 interval, void* buffstart, int bufflen);

UINT32 interval;     // サンプルインターバル

                     //(LIBSN.H が使用可能な常数を定義
                     //_1KHZ, _2KHZ, _4KHZ, _10KHZ, _20KHZ)

void* buffstart;     // プロファイル  サンプルバッファの開始アドレス

   int bufflen;      // 上記サンプルバッファの長さ
                     // (バイト)
```

留意点: プロファイラ機能を初期化するため、アプリケーションにコールする関数です。タイマーインタラプトをフックし、プロファイラバッファを設定、タイマーを実行します。この時点以降、カーネルコードからバッファは書き込まれるので、固定アドレス、常時使用可能である必要があります。

例:

```
static unsigned long ProfData[8192]      // long aligned,
                                         // 4K から 64K バイトまで

snProfInit( _4KHZ, ProfData, sizeof( ProfData ) );
```

注: OSAlarm,をお使いの場合は、OSSetPeriodicAlarm()や OSInitAlarm()を使ってアラームを作成する「前に」snProfInit をコールして下さい。

```
void snProfSetInterval(UINT32 interval);

UINT32 interval;     // サンプルインターバル

                     //(LIBSN.H が使用可能な常数を定義
                     //_1KHZ, _2KHZ, _4KHZ, _10KHZ, _20KHZ)
```

留意点: snProfInit()がサンプルインターバルを変更した後はいつでもコールできます。ヘッダファイルがいくつか有効なレートでお好きなインターバルに設定できます。このインターバルは四期バスクロックに定義されていますので、

```
samples_per_second = (162000000/4)/interval
```

または、サンプル周波数に応じてインターバルを算出します。

```
interval = (162000000/4)/samples_per_second
```

例:

```
snProfSetInterval((162000000/4)/ 40000 );

                     // サンプルレートを 40KHz に設定
```

```
void snProfSetRange(int profmask, void* startpc, void* endpc);

   int profmask;     // プロファイルマスク値 – サンプルのみ
                     // (flags & profmask) != 0 の時有効

   void* startpc;    // ここより下のサンプルでは
                     // 破棄

   void* endpc;      // ここより上のサンプルでは
                     // 破棄
```

留意点: マスク、フラグともデフォルト値は 1 です。デフォルトの PC 領域は 0 to 0xFFFFFFFF です。従って、変更しない限りは全てのサンプルが蒐集され、デバッガで使用可能になります。これらの値を変更すると、コードのどの部分をプロファイルし、どの部分を無視するかを指定できます。このフィルタが原因でサンプルの大部分が破棄されている場合は、サンプルバッファが充填されるまで若干時間を要します。従ってデバッガプロファイルペインのアップデートにも時間がかかります。これを補うにはバッファサイズを小さくし、サンプルレートを高くします。PC 領域にコードなしとする、あるいは指定したフラグ値をコールしないなど、全てのサンプルをフィルタアウトすると、プロファイルウィンドウがアップデートを停止します。

例:

```
// このモジュールのサンプルフラグ値を受領するよう
// プロファイリングを設定
```

```
snProfSetRange(-1, firstfuncinthismodule,
firstfuncinnextmodule);

extern int snProfSetFlagValue(int value);

int value;     // フラグのプロファイルはこの絶対値で
               // 設定される
```

留意点: プロファイラフラグは 32 の異なるブールコンディションで閲覧できます。この機能で、プログラムの現状をプロファイラで限定的にフィルタしたように設定することもできます。これらの特徴を使用し、例えば「プログラムでフラグ 4 が設定されている部分のプロファイル結果を表示」など、デバッガプロファイラペインでプログラムで特定の場所を設定できます。

注: 数字ではなく、論理バイナリフラグとして扱ってみて下さい。これはプロファイラがユーザ指定マスクの異なる位置使用するため、マスク値 3 がプロファイルフラグ値ビット 0 もしくはビット 1 で設定し、ビット 0 および 1 がゼロのサンプルは却下されます。

つまり、もし mask = 3 ならば、フラグ値は 1、2、3、5 などの処理が可能ですが、4 や 8 などのフラグ値は却下されます。

```
int snProfSetFlags(int flags);

int flags;     // 設定するフラグビット(32 の「コンディション」に
               // 32 ビット)
```

留意点:プロファイラフラグは 32 の異なるブールコンディションで閲覧できます。この機能で、プログラムの現状をプロファイラで限定的にフィルタしたように設定することもできます。この関数は追加部分にのみ設定できます。クリアすることはありません。フラグワード全てを絶対値として設定する際は snProfSetFlagValue()を代りに使用して下さい。

```
int snProfClrFlags(int flags);

int flags; // flag bits to clear (32 bits for 32 "conditions")
```

留意点:プロファイラフラグは 32 の異なるブールコンディションで閲覧できます。この機能で、プログラムの現状をプロファイラで限定的にフィルタしたように設定することもできます。この関数は追加部分のみクリアします。追加することはありません。フラグパラメータで 1 にセットされたビットはプロファイラの制御ブロックのフラグワードにクリアされます。フラグワード全てを絶対値として設定する際は snProfSetFlagValue()を代りに使用して下さい。

[このページは意図的に空白にされています]

別表 A: アセンブラ参考資料

サポートされていないアセンブラオプション

以下のアセンブラオプションは、使用しても認識されないか、エラーを起こすものです。

スイッチ	使用した場合の動作
-a	認識しない
-D	認識しない
-f	認識しない
--itbl	エラーが発生する
-J	認識しない
-K	認識しない
--listing-lhs-width	認識しない
--listing-lhs-width2	認識しない
--listing-rhs-width	認識しない
--listing-cont-lines	認識しない
-membedded-pic	エラーが発生する
-nocpp	認識しない
-R	エラーが発生する
--traditional-format	認識しない
-w	認識しない
-X	認識しない
-Z	認識しない

サポートされていないアセンブラダイレクティブ

以下の表は、現在 SN Systems のアセンブラがサポートしていない命令と、それを使用した場合の動作を示しています。

命令	使用した場合の動作
.abicalls	エラーが発生する

.cpadd	エラーが発生する
.cpload	エラーが発生する
.cprestore	エラーが発生する
.eject	認識しない
.endfunc	認識しない
.format	認識しない
.func	認識しない
.gpword	エラーが発生する
.ident	認識しない
.insn	認識しない
.lflags	認識しない
.linkonce	エラーが発生する
.list	認識しない
.livereg	認識しない
.llen	認識しない
.lsym	認識しない
.name	認識しない
.noformat	認識しない
.nolist	認識しない
.nopage	認識しない
.org	認識しない
.plen	認識しない
.rva	認識しない
.sbttl	認識しない
.spc	認識しない
.struct	認識しない
.title	認識しない

別表 B: リンカ参考資料

サポートされていない ld スイッチ

ngcld は、全ての標準的 ld コマンドラインを認識しますが、以下の非対応スイッチを使用すると、警告を表示します。

スイッチ	詳細
-aarchive	HP/UX 互換のための共有ライブラリ管理
-ashared	HP/UX 互換のための共有ライブラリ管理
-adefault	HP/UX 互換のための共有ライブラリ管理
-architecture	アーキテクチャを設定する
-A	アーキテクチャを設定する
-Bdynamic	共有ライブラリをリンクする
-Bstatic	共有ライブラリをリンクしない
-Bsymbolic	グローバルなレファレンスをローカルでバインドする
-call_shared	共有ライブラリをリンクする
-c	MRI 形式のリンカスクリプトを読み込む
-dy	共有ライブラリをリンクする
-dn	共有ライブラリをリンクしない
-dynamic-linker	動的リンカを使用できるよう設定する
-embedded-relocs	埋め込み再配置可能ファイルを生成する
-EB	ビッグエンディアンオブジェクトをリンクする
-format	次に続く入力ファイルに出力先を指定する
-filter	共有のオブジェクトシンボル表をフィルタリングする
-force-exe-suffix	`.exe` 拡張子がついたファイルを強制的に生成する
-f	共有のオブジェクトシンボル表をフィルタリングする (予備)
-F	共有のオブジェクトシンボル表をフィルタリングする
-h	共有ライブラリの内部名を設定する
-mri-script	MRI 形式のリンカスクリプトを読みこむ

-m	エミュレーションを設定する
-non_shared	共有ライブラリにリンクを設定しない
-oformat	出力ファイルの出力先を指定する
-relax	特定の出力先の分岐を広げる
-rpath dir	ランタイムライブラリの検索パスにディレクトリを追加する
-rpath-link DIR	指定ディレクトリにある必要な共有ライブラリ の場所を検索する
-soname	共有ライブラリの内部名を設定する
-split-by-file	出力セクションを各ファイルに分割する
-split-by-reloc	出力セクションを各 COUNT 再配置可能ファイルに分割する
-shared	共有ライブラリを作成する
-task-link	タスクレベルのリンクを実行する
-traditional-format	ネイティブのリンカと同じフォーマットを使用する
-version-script	バージョン情報スクリプトを読む
-wrap	SYMBOL のラッパー機能を使用する

ngcld でサポートしていないスクリプトファイル命令

以下のスクリプトファイル命令を使用すると、警告が表示されます。

```
OUTPUT_ARCH
OUTPUT_FORMAT
```

以下のスクリプトファイル命令は、ngcld が対応していないため、使用するとエラーが発生します。

```
HLL
SYSLIB
VERSION
TARGET
```

以下の命令は受理されますが、認識しません。

```
CONSTRUCTORS
NOCROSSREFS
MEMORY
SORT
KEEP
CREATE_OBJECT_SYMBOLS
```

エラーメッセージ

ProDG リンカは番号で表されたエラーメッセージを表示します。一部はすべてのコンソールにビルドされたリンカに共通なメッセージですが特定のプラットフォームにのみ適用されるものもあります。

共通エラーメッセージ

次のエラーはすべての ProDG リンカによって生成されると考えられます。

エラー	説明
L0001	"<FILE>"から読み取りの際に ReadFile()不成功
L0002	SetFilePointer("<OFFSET>")不成功
L0003	SetFilePointer(<OFFSET>,0,NULL,FILE_CURRENT)不成功
L0004	'<SYMBOL>'シンボルは<FILE>(<OBJECT>)の中に見当たらない
L0005	想定外のトークン
L0006	WriteFile()不成功
L0007	-N オプションはサポートされていない
L0008	-omagic はサポートされていない
L0009	-n はサポートされていない
L0010	-nmagic はサポートされていない
L0011	-export_dynamic_symbols はサポートされていない
L0012	-E はサポートされていない
L0013	.line インフォメーションを<FILE>のファイルに読み取ることに失敗
L0014	<FILE>のファイルの Elf sym テーブルに<SYMBOL>を発見できなかった
L0015	システムエラー <NUMBE>:<MESSAGE>
L0016	<SYMBOL>のシンボルはスモールデータセクションでは定義されていないが short reference されていることはありうる
L0017	<FILE>からの”<SYMBOL>”のシンボルへの short reference
L0018	<FILE>からの”<SYMBOL>”のシンボルへの short reference
L0019	'<SYMBOL>'のシンボルが複数定義されている
L0020	共通シンボルの<SYMBOL>が複数定義されている
L0021	グループの終了をしないで二番目のグループの開始をしている
L0022	SORT(CONSTRUCTORS) KEEP オペランドとして無効 – 無視
L0023	OUTPUT_FORMAT 無視

L0024	OUTPUT_ARCH 無視
L0025	リンカスクリプトに多数のセクションが発見されていない、例<SECTION NAME>
L0026	"<SECTION NAME>"に類似したセクションを発見できない。リンクの終わりに追加
L0027	デバグファイルが二度特定されている。"-debug <FILE>"は無視
L0028	<OPTION>オプションが直接リンカに送られている
L0029	グループの開始の無いグループの終わり
L0030	Bss 属性の無視
L0031	スクリプトファイルの<FILE>を発見できない
L0032	<FILE>のファイルを開くことができない
L0033	入力地点のラベルの<SYMBOL>は定義されていない
L0034	"<SECTION NAME>"のセクションを出力中のエラー
L0035	ファイルの特定後、(が必要
L0036	コマンドラインのファイルの<FILE>を開くことが出来なかった
L0037	{ が無い
L0038	入力ファイルが無い
L0039	<FILE>のファイルの中の未定義シンボルの<SYMBOL>への参照をしている
L0040	コードを含んでいる入力オブジェクトが無い
L0041	書式エラー
L0042	表現に書式エラー有り
L0044	<NAME> は認識されたプログラムヘッダではない
L0045	<FILE>の中の 0x<NUMBER>は不良パッチタイプ
L0046	ローカルシンボルの<SYMBOL>に参照を作れない
L0047	グローバルシンボルの<SYMBOL>に参照を作れない
L0048	Elf ファイルの '<FILE>'を開くことができない
L0049	<FILE>の中で未定義のセクション番号 0x<NUMBER> への参照
L0050	バッファ外への書き込みの試み
L0051	未知の再配置タイプ
L0052	Lib の読み取りエラー。Lib の<FILE>のモジュール名の"<MODULE>"に"/"が必要。
L0053	<FILE>のライブラリの中にシンボルテーブルを見つけられない

L0054	<FILE>のライブラリの中に必要とされる文字列テーブルを見つけられない
L0055	出力ファイルの<FILE>を見つけられない
L0056	<NUMBER>は未知のシンボルタイプ
L0057	デバッグ出力ファイルの<FILE>を開けられない

ngcld に特有のエラーメッセージ

次の表は ngcld リンカに特有のエラーメッセージを表記しています。

エラー	説明
L1000	小データオフセットの 0x<ADDRESS>からアドレス<ADDRESS>が範囲外。
L1001	アドレス 0x<ADDRESS>への小データ参照の作成に失敗した。
L1002	サイズ<SIZE>の共通シンボル"<SYMBOL>"は SDA に入れられた。
L1003	加数を利用した再配置が期待されている。
L1004	エラーをレポートするためにデバッグ情報を読み出すことができない。

[このページは意図的に空白にされています]

インデックス

blankelf - blank ELF ユーティリティ 75
C プログラミングに関する考慮 35
C と C++でのメモリ管理 21
ELF のサイズ最適化 65
ELF ファイルを起動/実行する 14
GameCube プロジェクトをビルドする 13
Gekko プロファイラ API 85
Gekko プロファイリング用ビルド 85
GNU ビルド ツール 6
I/O ストリームバッファを使用する 24
libsn.a - SN Systems ライブラリ 7
make 75
malloc と free を使ったプログラミング 22
naked 関数のサポート 35
new と delete を利用した C++のプログラム 23
ngccc オプション 28
ngccc コマンドライン書式 28
ngccc コンパイラ ドライバの使用 27
ngcld
  コマンドライン書式 60
  スイッチ 61
  リンカを使用する 59
  小データ領域のサポート 62
ngcld でサポートしていないスクリプトファイル命令 92
ProDG ビルド ツール 5
ProDG ユーティリティ 5
ProDG コンパイラドライバ
  ゲームキューブライブラリを指定する 34
ProDG デバッガとの統合 15
SN API for PS (SNAPS) 演算 39
SN Systems へのフィードバック 70
SN Systems のアセンブラを使用する 49
SN Systems 命令 51
SN.INI ファイル 6
snarl - SN アーカイヴライブラリアン 76
Visual C++ がブラウズ情報を生成する仕組み 17
Visual C++ はどのようにして依存情報を生成するか 16
Visual Studio 6.0 統合機能の概要 9
Visual Studio 6.0 統合機能を設定する 12
Visual Studio 統合機能
  テンポラリファイルを削除する 16
  ブレークポイント 15
  プロジェクトファイルを追加する 13
  編集ソース 16
Visual Studio で GameCube プロジェクトを作成する 9
Web でのアップデート 2
アクティブプロジェクトの設定 11
アセンブラ命令 50
エラーおよび警告メッセージ 54
エラーメッセージ 93
グローバル デストラクタの削除 67
ゲームキューブプログラミングのヒント 45
コマンドライン書式 49
コンパイラ オプション 65
コンパイラ ドライバ
  応答ファイル 35
コンパイラの最適化スイッチの使用方法 68

サポートされていない ld スイッチ 91
サポートされていないアセンブラオプション 89
サポートされていないアセンブラダイレクティブ 89
システム リソースの最大化 69
シンボル名 54
シンボル情報の削除 67
セクション 60
ソースのブラウジング 17
テクニカル サポートとアップグレード 2
テクニカルサポート 70
デフォルトファイル拡張子アクションを最優先する 34
テンポラリファイルを削除する 16
トラブルシューティング 18, 46, 64
ビルディング アプリケーション 6
ビルドの最適化 65
ビルド速度の最適化 68
ファイル名 33
ファイル名と行番号 53
ブレークポイント 15
プロジェクトファイルを追加する 13
プロジェクトをビルドする 13
リンカ オプション 66
リンカ スクリプト 59
リンカスクリプトでファイルを参照する 60
レジスタリファレンス 55
レジスタ名称指定およびレジスタフィールド 51
ローカルシンボル 55
一般的なプログラミングにおける考慮事項 41
不必要なインクルード ファイルの排除 68
不必要なデバッグ情報の削除 67
使用 27
依存情報の生成 16
入力ファイル 53
再配置可能モジュールをビルドする 71
出力(オブジェクト)ファイル 54
単一ペア浮動小数点演算 37
実行速度向上のための最適化 68
必要メモリ 59
応答ファイル 35
概要 1
範囲の指定 52
編集ソース 16
重複する非インライン関数の生成防止 67
高速キャスティング 36